

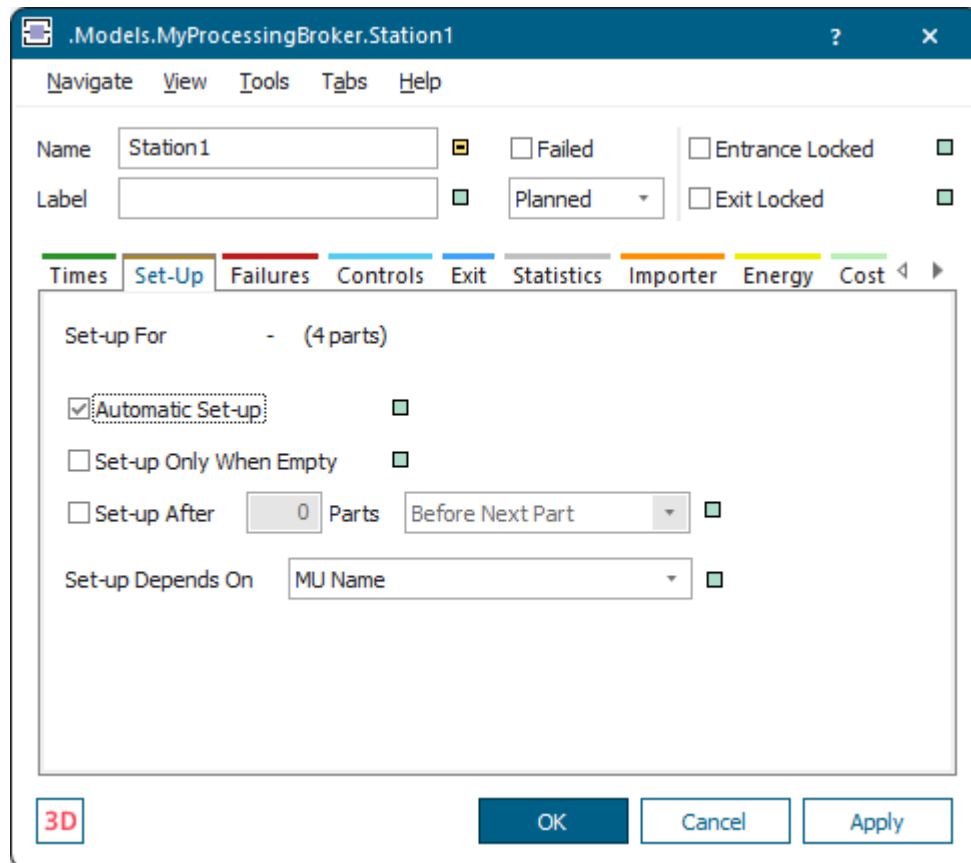
Setting a Station Up

Set a Station Up

You can define how to set up the objects *Station*, *ParallelStation*, *AssemblyStation*, *DismantleStation*, and *Drain* to process another type of *MU*.

You can:

- **Select Set-up Options** on the tab **Set-Up**.
- **Select the Set-Up Time** on the tab **Times**.



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

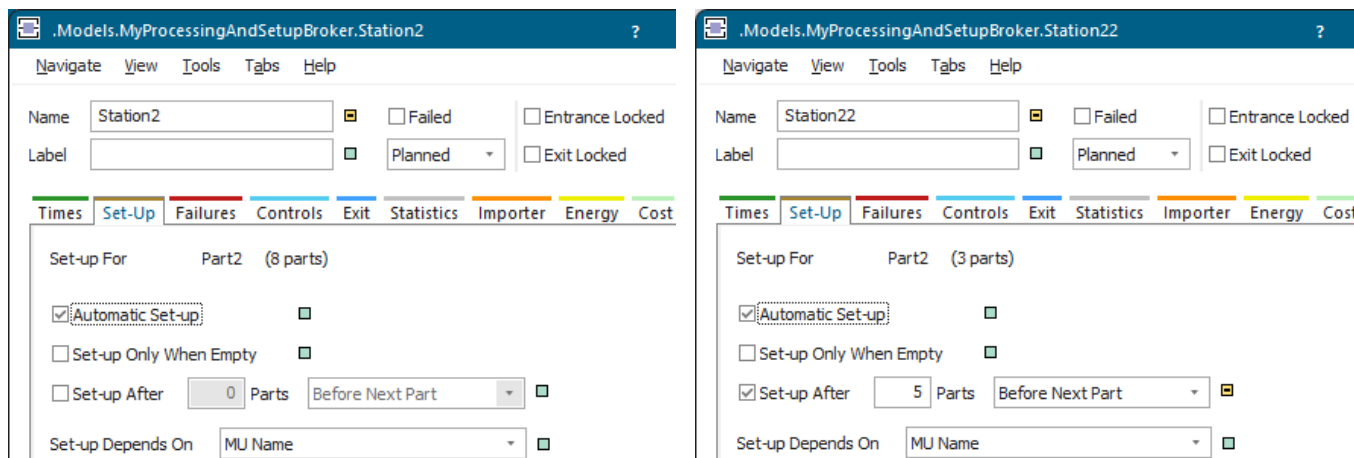
Back to [Model the Flow of Materials, Basics](#)

Selecting Set-up Options

Select Set-up Options

When you set up a station you will first select the set-up options. Next to **Set-up For Plant Simulation** show which type of *MU* the object is presently set up for or is going to be set up for.

If the object has not been set up yet at all, it shows a hyphen -. To tell one type of *MU* from the other, *Plant Simulation* either uses the name of the *MU* or a user-defined attribute of the *MU*.



First, you will select if you want to:

- [Set the Station Up Automatically](#)
- [Only Set the Station Up When it is Empty](#)
- [Set the Station Up after it Processed a Certain Number of Parts](#)
- After you have done this, you will [Select the Set-Up Criteria](#).

Go to [Select the Set-Up Time](#)

Back to [Set a Station Up](#)

Set the Station Up Automatically

To automatically set the object up, select **Automatic**. Then, the material flow object triggers the set-up process as soon as a *MU* of the corresponding *MU* type intends to move onto it.

Only Set the Station Up When it is Empty

The screenshot shows the 'Set-Up' tab of the 'Station2' configuration window. The 'Name' field is 'Station2' and the 'Label' field is empty. The 'Planned' dropdown is selected. The 'Set-up For' field is 'Part2 (8 parts)'. The 'Automatic Set-up' checkbox is checked, and the 'Set-up Only When Empty' checkbox is unchecked. The 'Set-up After' field is '0 Parts' and the 'Before Next Part' dropdown is selected. The 'Set-up Depends On' field is 'MU Name'.

To define the set-up process in a *Method*, clear the check box.

Back to [Select Set-up Options](#)

Only Set the Station Up When it is Empty

To only start setting-up the object for the next type of *MU* when it is **Empty**, select **Only When Empty**. *MUs* of the type for which the object is setting-up can only enter the object, after the set-up process is finished.

The screenshot shows the 'Set-Up' tab of the 'Station2' configuration window. The 'Name' field is 'Station2' and the 'Label' field is empty. The 'Planned' dropdown is selected. The 'Set-up For' field is 'Part2 (8 parts)'. The 'Automatic Set-up' checkbox is checked, and the 'Set-up Only When Empty' checkbox is unchecked. The 'Set-up After' field is '0 Parts' and the 'Before Next Part' dropdown is selected. The 'Set-up Depends On' field is 'MU Name'.

To allow *MUs* to enter the object although it has not been set-up for their type yet, clear the check box.

Their set-up time begins, when all set-up processes you specified are finished. *MUs* located on the object will be processed with the settings for the *MU* type for which the object was set-up before.

See also [Empty \[state, material flow objects\]](#)

Back to [Select Set-up Options](#)

Set the Station Up after it Processed a Certain Number of Parts

To set the material flow object up, after a certain number of parts have been processed, select the check box **Set-up After**. Then, enter the number of parts into the text box after which you want the object to be set up.

The object also sets up and starts counting anew, starting from 1, when the part type changes before the number you enter is reached. For this reason it may happen that the object does not reach the number of parts of a certain type for which it is to set up.

Note

The *ParallelStation* does not provide the setting [Set-up After n Parts](#).

The object always sets up for the part type, which is going to enter the object next. For this reason the set-up process does not start immediately after the *n*-th part has been processed or when this part exits the object. Setting-up starts as soon as the (*n*+1)-th part enters the station.

- Select **Before Next Part** to set the object up immediately after the *n*-th part was processed.
- Select **After Last Part** to set the object up after the *n*-plus-first part wants to move onto the station.

Note

The number of parts is the number of parts entering the station. These parts do not necessarily have to be processed. Statistics also counts parts, which are removed from the station before they have been processed, for example during the set-up process or while the station is waiting for a service to be performed.

Back to [Select Set-up Options](#)

Select the Set-Up Criteria

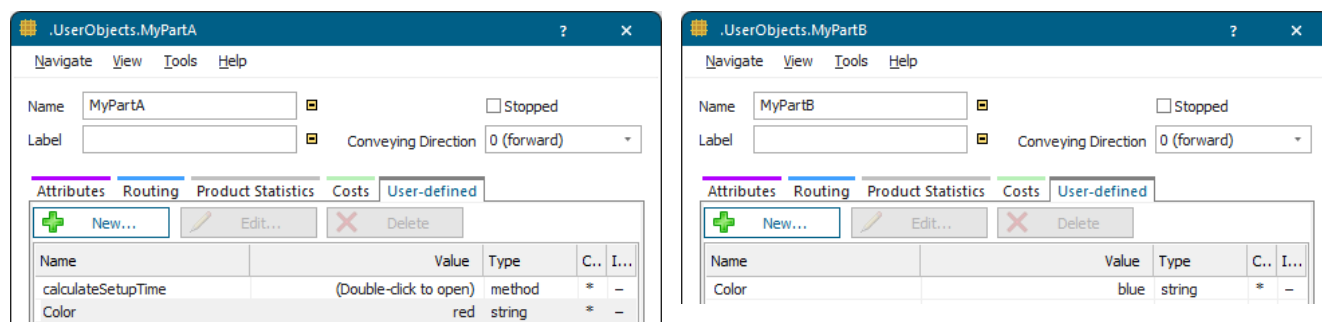
After you decided how the station sets-up, you can select when the object has to be set up.

- When the name of the *MU* (**MU Name**), which wants to move onto the station, changes.
- When the value of a **user-defined attribute of the MU** changes.

The screenshot shows the configuration window for a station named 'Station12'. The 'Set-Up' tab is active. Under 'Set-up For', it is set to 'Part2 (4 parts)'. The 'Automatic Set-up' checkbox is checked. The 'Set-up Depends On' dropdown is set to 'User-defined Attribute', and the text box below it contains 'Color'. Other options like 'Set-up Only When Empty' and 'Set-up After' are unchecked.

- Select **User-defined Attribute** from the drop-down list.
- Enter the name of the user-defined attribute of the *MU* into the text box. This user-defined attribute has to be of data type *string*. The material flow object sets up, when a *MU*, whose user-defined attribute has a different value, moves onto the station.

In our example we created a user-defined attribute named **CoLoR** for the *MU* types. Let us assume that the station is set up for red. When a *MU* with the user-defined attribute **Value bLue** moves onto the station, it sets up for **bLue**.

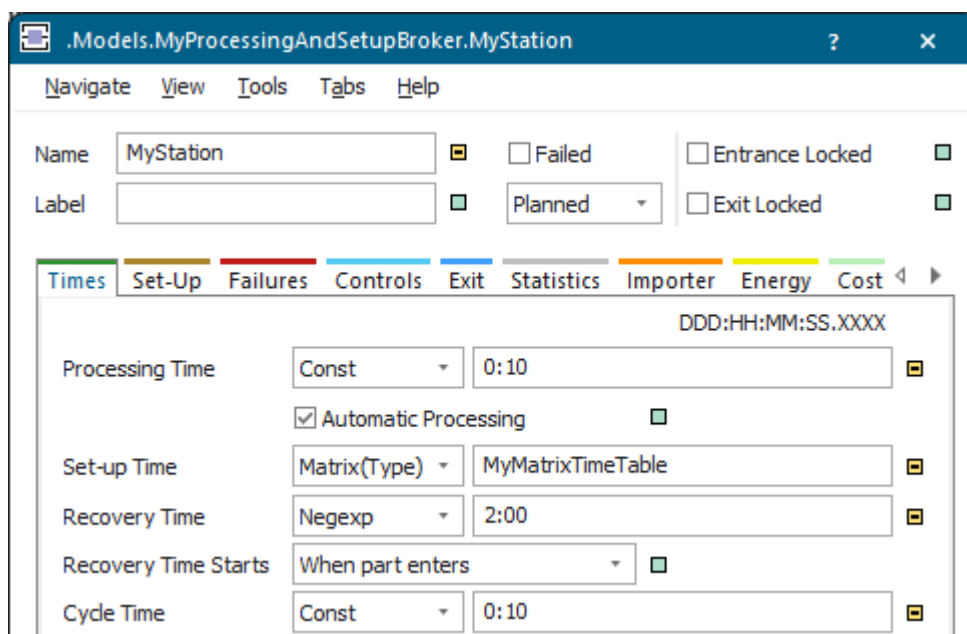


Back to Set a Station Up

Select the Set-Up Time

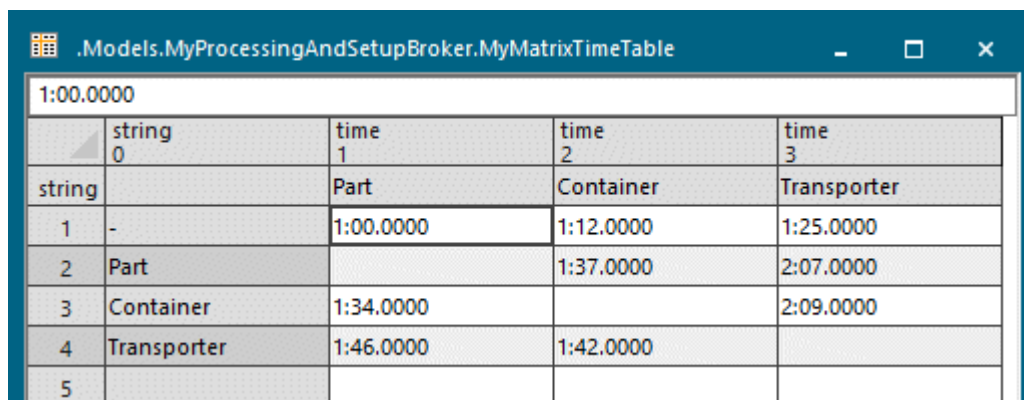
The **Set-up Time** is the time it takes to set the station up for processing a different type of *MU*. An identical name designates that *MUs* are of the same type. In this case the material flow object will not have to set up for a different *MU* type. It only needs to set up, when the *MUs* are of another type, denoted by a name that differs from the previous type.

For the **Set-up Time**, you can select a distribution from the drop-down list on the tab **Times**, depending on the type of *MU*, or the location on a station, or you can enter a constant time. When you selected a distribution, you will then enter the values, which that distribution requires, into the text box. *Plant Simulation* show these values along the upper border of the tab. In addition to any of the other distributions, you can select the **Matrix(Type)** distribution.



When setting-up, the time may not only depend on the target type, for which you want to set up, but also on the source type. In this case you can define the times in a table. Activate the user-defined row index and

the user-defined column index of the table. The row index designates the source type. The column index designates the target type.



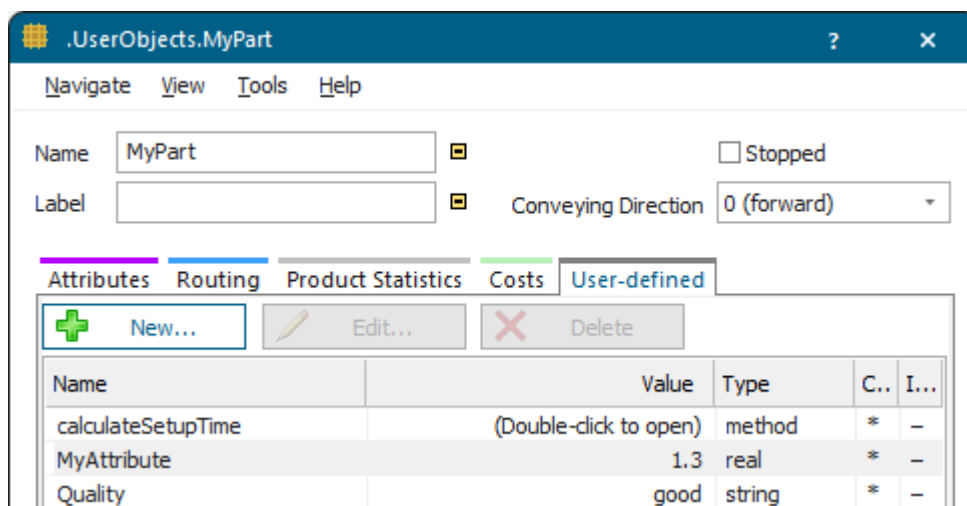
	string 0	time 1	time 2	time 3
string		Part	Container	Transporter
1	-	1:00.0000	1:12.0000	1:25.0000
2	Part		1:37.0000	2:07.0000
3	Container	1:34.0000		2:09.0000
4	Transporter	1:46.0000	1:42.0000	
5				

In our example above setting-up from no type, indicated by the hyphen, to the target type *Part* takes exactly one minute. Setting-up from the type *Part* to the type *Transporter* takes two minutes and seven seconds.

If you select the **Formula** distribution, you can enter a numeric expression or the name of a *Method*. You can use the anonymous identifier @ to access the *MU* for which the set-up time applies.

Note

You can also determine the set-up time in a user-defined attribute of data type *method*, which you created for the **MU**!



Name	Value	Type	C..	I...
calculateSetupTime	(Double-click to open)	method	*	-
MyAttribute	1.3	real	*	-
Quality	good	string	*	-

Compare Specify Times in Object Dialogs

Go to Select Set-up Options

Back to [Setting a Station Up](#)

Defining Processing Times

Define Processing Times

The **Processing Time** is the time, which the *MU* remains on the material flow object to be processed. It is the interval between set-up and the time the material flow object moves it on to its successor.

The screenshot shows the configuration window for a station named 'MyStation'. The 'Times' tab is active, displaying various time-related settings. The 'Processing Time' is configured with a 'Uniform' distribution and a range of '0:30, 1:10'. The 'Set-up Time' is set to 'Matrix(Type)' with the table 'MyMatrixTimeTable'. The 'Recovery Time' is set to 'Negexp' with a value of '2:00'. The 'Cycle Time' is set to 'Const' with a value of '0:10'. There are also checkboxes for 'Automatic Processing' and 'Entrance Locked', and a dropdown for 'Recovery Time Starts' set to 'When part enters'.

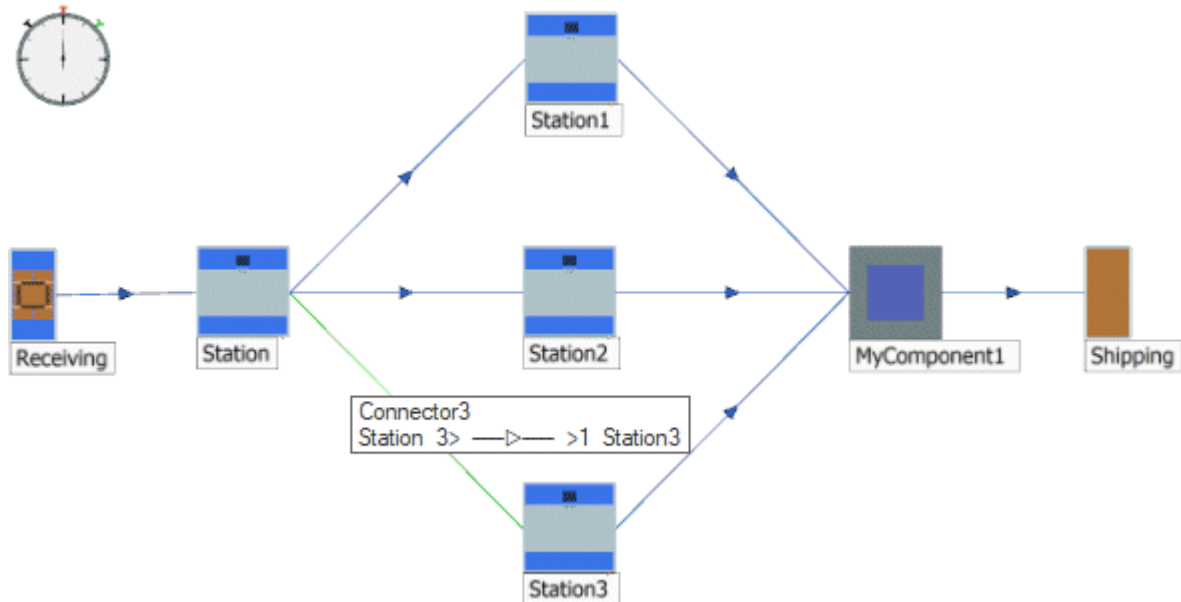
On the tab **Times** you can:

- Select a distribution from the drop-down list and enter the required values
 Processing Time Uniform 0:30, 1:10
- Enter a constant time (**Const**). In our example we entered half a minute (30 seconds)
 Processing time: Const 0:30
- Select **List(Type)** to process the *MU* depending on its type
 Processing Time List(Type) MyListTypeTable
- Select **List(Place)** to process the *MU* depending on the station on which it is located on a
 ParallelStation Processing Time List(Place) MyListPlaceTable

After you selected a distribution, you will then enter the values, which that distribution requires, into the text box. *Plant Simulation* shows these values along the upper border of the tab. For the negative exponential distribution for the **Recovery Time** in the example above, you have to enter values for **Beta**, and optionally the **Lower Bound** and the **Upper Bound**.

When you select **Formula**, you can type in a numeric expression or the name of a *Method*. You can use the anonymous identifier **@** to access the *MU* for which the processing time applies. This *Method* may also be a *user-defined attribute* of data type *method* of the *MU*.

The successor is the object that is connected to the selected object with a *Connector* and that succeeds it in the sequence of stations in the simulation model.



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

Compare [Specify Times in Object Dialogs](#)

Compare [Change the Processing Time in the Entrance Control](#)

Compare [Set a Station Up](#)

Compare [Use Probability Distributions](#)

Back to [Model the Flow of Materials, Basics](#)

[Video on YouTube](#)

<https://youtu.be/PQhEriOzVzU?si=gwlvKoQ0I2mro1IB&t=31>

Specify Times in Object Dialogs

Plant Simulation inputs and outputs data referring to times in the format 1:00:00:00, standing for, left to right, days:hours:minutes:seconds. split seconds. 12:34 for example, means 12 minutes and 34 seconds.

DDD:HH:MM:SS.XXXX

Processing Time

Const

0:12

If you do not want to write one day out in full, just type 1: : : and click **Apply**

Apply

. *Plant Simulation* then translates this to the full format 1:00:00:00.

1 day	1 hour	1 minute	1 second	1.5 seconds
Processing time: Const 1:::	Processing time: Const 1::	DDD:HH:MM:SS.XXXX Processing time: Const 1:	DDD:HH:MM:SS.XXXX Processing Time: Const :01	DDD:HH:MM:SS.XXXX Processing Time: Const :01.5
Processing time: Const 1:00:00:00	Processing time: Const 1:00:00	DDD:HH:MM:SS.XXXX Processing time: Const 1:00	DDD:HH:MM:SS.XXXX Processing Time: Const 0:01	DDD:HH:MM:SS.XXXX Processing Time: Const 0:01.5

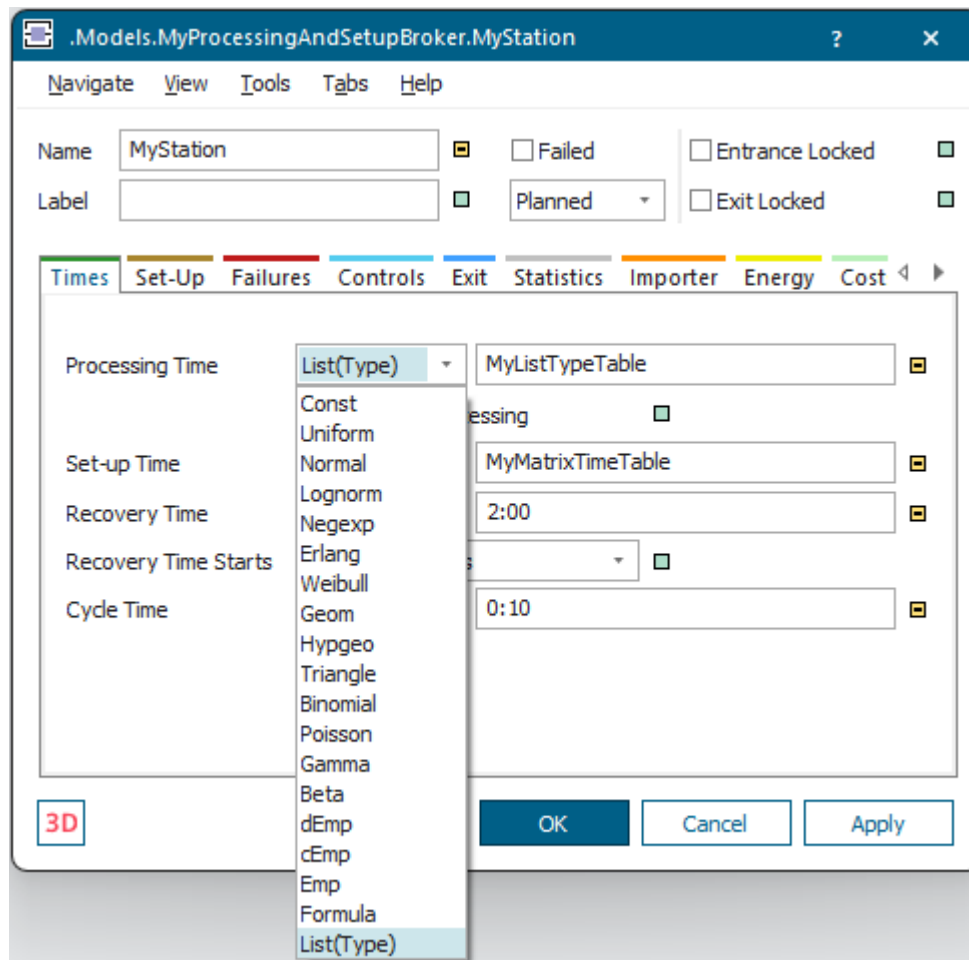
You can also enter numbers without the colon. *Plant Simulation* then interprets the number as seconds and converts it into the above format. 111 (seconds), for example, is 1 minute and 51 seconds.

You can change time-related settings under **File > Model Settings/Preferences > Units > Time Scale**.

Back to Times

Specify Data of a Probability Distribution

Select the distribution you want to use for the **Processing Time** from the drop-down list on the **Tab Times**. Then, *Plant Simulation* shows the parameters, which this distribution requires, when you click into the text box.



Enter the respective values into the text box. The lower bound and the upper bound are optional, you can, but do not have to specify them.

.Models.MyProcessingAndSetupBroker.MyStation

Name: MyStation
 Label:
 Failed: ☐ Entrance Locked: ☐
 Planned: ☐ Exit Locked: ☐

Times | Set-Up | Failures | Controls | Exit | Statistics | Importer | Energy | Cost

Processing Time: Uniform | Lower Bound, Upper Bound: 0:30, 1:10
☒ Automatic Processing

Set-up Time: Matrix(Type) | MyMatrixTimeTable

Recovery Time: Negexp | 2:00

Recovery Time Starts: When part enters

Cycle Time: Const | 0:10

Compare [Tab Times \[general description\]](#)

Compare [Use Probability Distributions](#)

Back to [Define Processing Times](#)

Define Processing Times Depending on the Type of MU

To set the **Processing Time** of a station depending on the type of *MU*, you can use the **List(Type)** distribution.

.Models.MyProcessingAndSetupBroker.MyStation

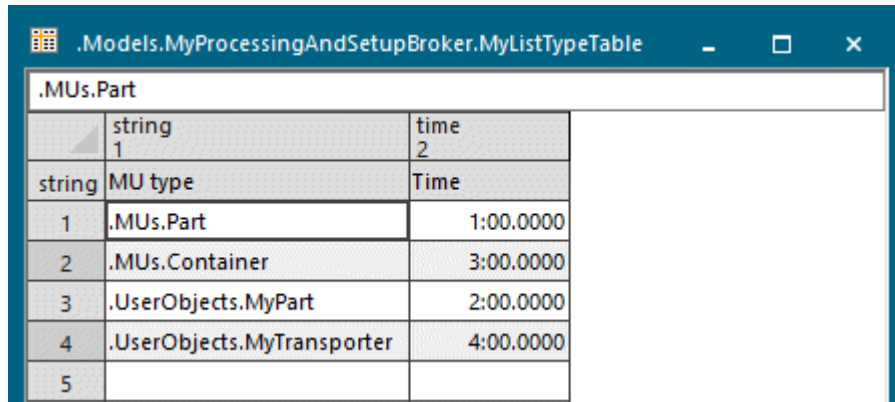
Name: MyStation
 Label:
 Failed: ☐ Entrance Locked: ☐
 Planned: ☐ Exit Locked: ☐

Times | Set-Up | Failures | Controls | Exit | Statistics | Importer | Energy | Cost

Processing Time: List(Type) | Table[string,[string,string,]time]: MyListTypeTable
☒ Automatic Processing

- Select the **List(Type)** distribution as the **Processing Time**.
- Enter the name of the *DataTable* object into the text box.

Enter the names of all *MUs* to be processed into column 1 of the table and the respective times in seconds into column 2. *Plant Simulation* then uses this time as the processing time for the respective *MU* type.



	string 1	time 2
	string MU type	Time
1	.MUs.Part	1:00.0000
2	.MUs.Container	3:00.0000
3	.UserObjects.MyPart	2:00.0000
4	.UserObjects.MyTransporter	4:00.0000
5		

During the simulation run *Plant Simulation* reads the processing time from that table.

Back to Define Processing Times

Define Processing Times in a Formula

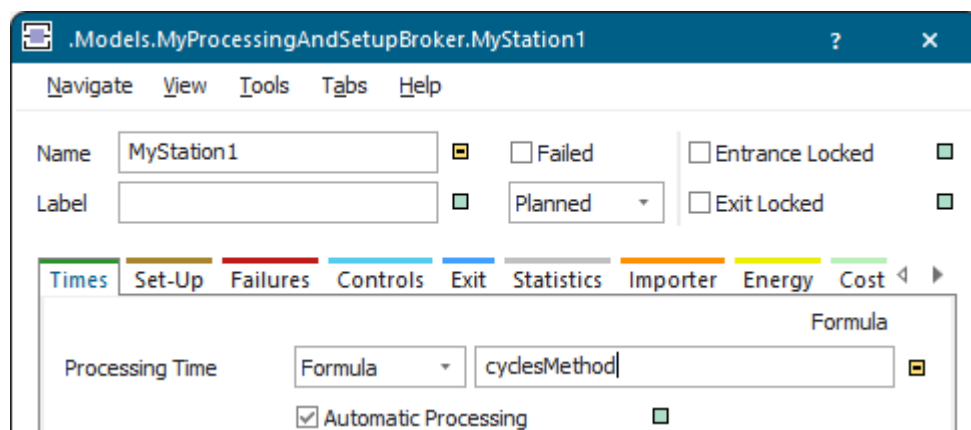
You can define the **Processing Time** of the objects in a **Formula**.

To define the **Processing Time** of a station in a formula:

- Type the name of a *Method* into the text box and program the formula in that *Method*.
- Type the formula as a basic arithmetic operation directly into the text box.
- Type the formula directly into the text box and access the *MU* for which the processing time applies with the anonymous identifier @.

To set the processing time of a station in a formula using a **Method**:

- Select **Formula** as the **Processing Time**.



MyStation1

Name: MyStation1

Label:

Failed: ☐ Entrance Locked: ☐

Planned: ☐ Exit Locked: ☐

Times | Set-Up | Failures | Controls | Exit | Statistics | Importer | Energy | Cost

Processing Time: Formula

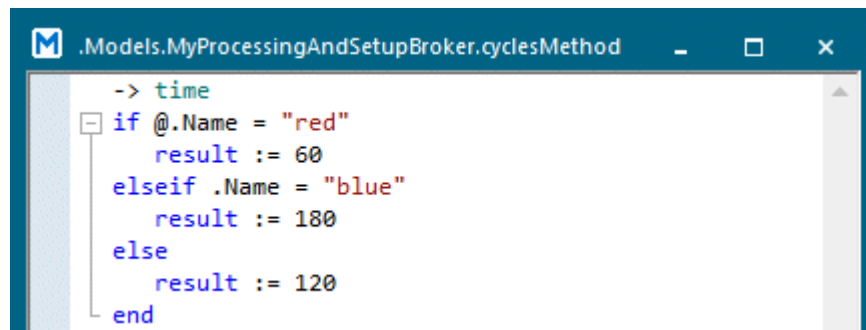
Formula: cyclesMethod

☒ Automatic Processing

- Enter the name of a *Method* object into the text box. Type the formula into the *Method*. This *Method* has to return a value of data type *time*.

The source code looks like this:

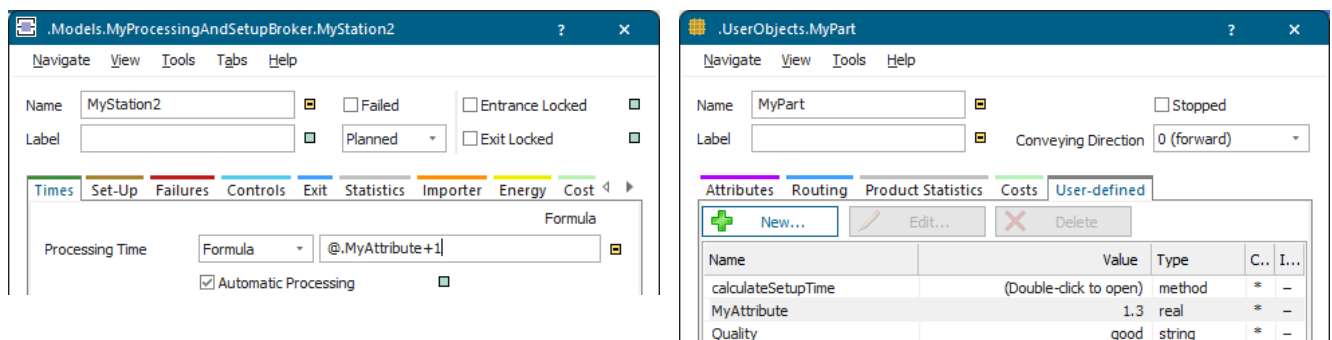
```
-> time
if @.Name = "red"
    result := 60
elseif .Name = "blue"
    result := 180
else
    result := 120
end
```



In our example, the formula, which we typed into the method *cyclesMethod*, sets how long the station processes the *MUs* in accordance to their color.

To set the processing time of a station in a formula with an arithmetic operator:

- Select **Formula** as the **Processing Time**.
- Type the expression into the text box. In our example we typed `@.MyAttribute+1`. This formula adds 1 minute to the **Processing Time** of *MUs* that have the user-defined attribute *MyAttribute*. We defined *MyAttribute* in the class of the *MU*.



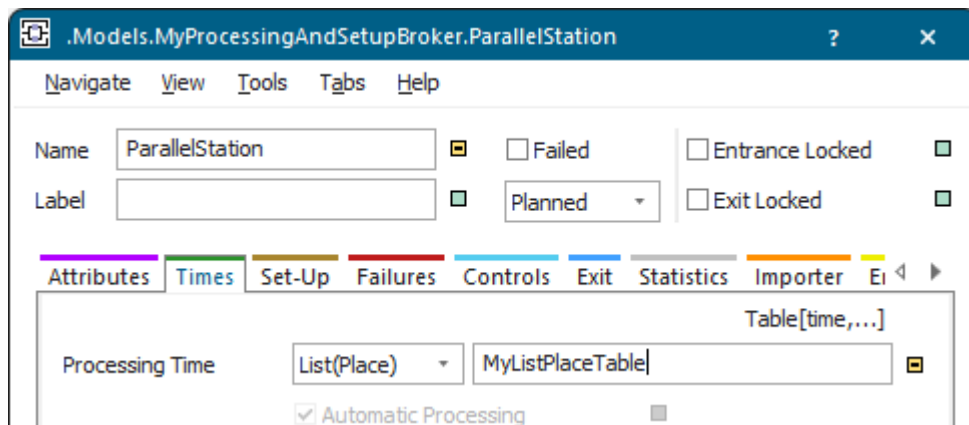
To set the processing time of a station by accessing the MU:

- Type `@.timeRed` into the text box.
- Create a user-defined attribute for the *MU* and name it `timeRed`.

Back to [Define Processing Times](#)

Define Processing Times of a ParallelStation

To set the **Processing Time** of a *ParallelStation* with several processing places, which have different but constant processing times, you can use the **List(Place)** distribution.



- Select the **List(Place)** distribution as the **Processing Time**.
- Enter the name of the *DataTable* object into the text box.

In this table, the entry `[1, 2]` matches the processing place located at position `[1, 2]`, for example.

When a *MU* arrives at a processing place of a *ParallelStation* during a simulation run, *Plant Simulation* takes the appropriate time for that station from the table.

time 1		time 2	
1	1:00.0000	10:00.0000	
2	1:20.0000	0.0000	

See also [ParallelStation](#)

Back to [Define Processing Times](#)

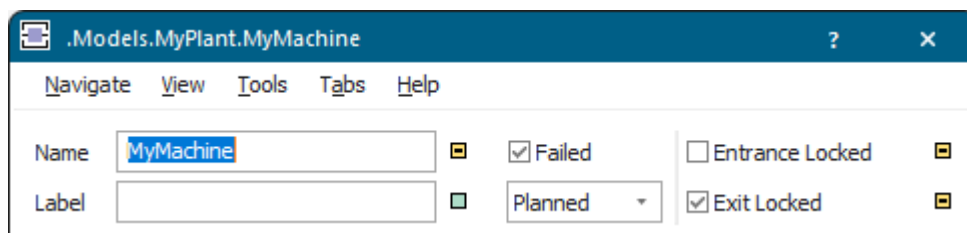
Modeling Failures

Model Failures

To closely model real situations, where machines fail at times, you can define one or several *failure profiles*. Failures do affect the technical availability of the individual stations.

You can:

- Manually fail the object by selecting **Failed** in the dialog of the material flow object. When you fail the object like this, you will also have to manually remove the failure by clearing the check box  **Failed** again.



- Define failures with the failure generator on the **Tab Failures**.

The state of the station then changes from *operational* to *failed*.

By default a **Failed** object shows its **state** with a red colored ring on a pole.

When you manually fail a station, it remains failed while any of the *failure profiles* you defined is active. It will change to not failed when the last failure (**DisruptionEnd**) of the last *failure profile* is over or you clear the check box  **Failed**.

As soon as the failure starts, the object changes to inactive for the duration of the failure. During this time it will not receive any parts. If a part is located on the object, its processing is interrupted for the duration of the failure and continues when the failure is cleared. *Plant Simulation* adds the duration of the failure to the processing time or to the dwelling time.

If a *MU* could not enter the object because of a failure, *Plant Simulation* unblocks the *MU* as soon as the failure ends.

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

Go to [Configure Failures](#)

Go to [Change Failure Settings During the Simulation Run](#)

Back to [Basics](#)

Configure Failures

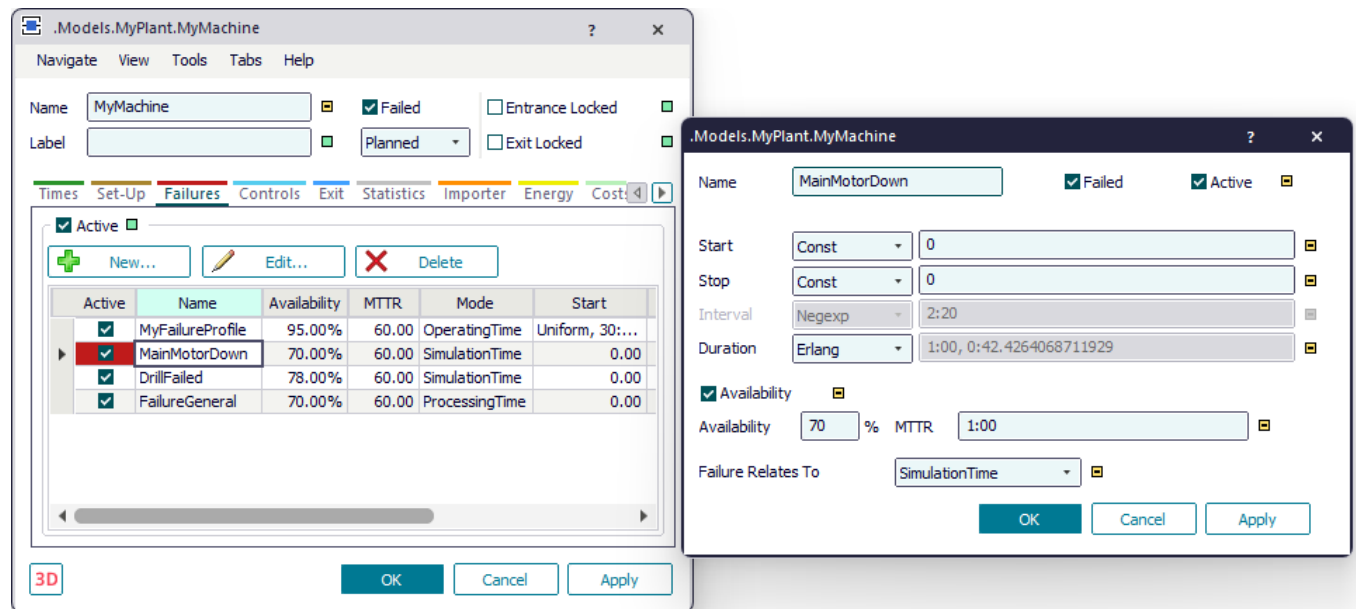
To simulate the real behavior of machines, which fail now and then and which have to be repaired, you can define one or several *failure profiles* for a station.

Note

If you change failure settings, we recommend to first clear the check box **Active** and to then click **Apply**. Then change your settings, apply them, and select the check box **Active** again. This ensures that the next **failure event** (**DisruptionBegin/DisruptionEnd**) will be computed with a complete set of valid parameters.

Note

The failure of the object is only active if you activate the check box **Active** on the tab **Failures** and the check box **Active** of the corresponding *failure profile*!



Procedure

1. To run your simulation model with failures in general, make sure that the check box **Active** on the **Tab Failures** is selected.
2. Click **New** and select and enter the parameters of the *failure profile* you are defining into the dialog that opens.
3. Enter a **Name** for the *failure profile*.
4. Select if this *failure profile* is **Failed** or not during the simulation run.
5. Select a distribution for the time at which the first failure takes place from the drop-down list **Start**.

Enter the values, which that distribution requires, into the text box. *Plant Simulation* shows these values along the upper border of the tab.

The **Lognormal distribution**, the **Erlang distribution**, and the **Negative exponential distribution** are especially suited for modeling failures.

6. Select a distribution for the time at which the last failure will take place from the drop-down list **Stop**. Enter the values, which that distribution requires. *Plant Simulation* shows these values above the list of distributions which you can select.

If you do not enter a **Start** time and a **Stop** time, the first failure occurs after the **Interval** you entered is over. Any value you enter as the **Start** time overwrites this behavior.

7. Select the check box **Availability** and enter the **Availability** in percent and **MTTR**. To specify the **Interval** and the **Duration** of a failure instead, clear the check box **Availability**. **Availability** and **MTTR** is just another kind for displaying the **Interval** and the **Duration**. If you enter values for **Availability** and **MTTR** and click **Apply**, *Plant Simulation* computes the values for the **Interval** and the **Duration** and enters them into the respective text boxes. It also selects the **Negexp** distribution for the **Interval** and the **Erlang** distribution for the **Duration**. *Plant Simulation* shows them in the dialog if you clear the check box **Availability**. An availability of 100 percent has an MTTR of 0, as the machine is available and does not have to be repaired.

The screenshot shows the 'Models.MyPlant.MyMachine' dialog box. It contains the following fields and controls:

- Name:** MyFailureProfile
- Failed:** ☐ **Active:** ☒
- Start:** Uniform (dropdown), 30:00, 35:00 (text box)
- Stop:** Const (dropdown), 0 (text box)
- Interval:** Negexp (dropdown), 19:00 (text box)
- Duration:** Erlang (dropdown), 0:59.2782730020053, 0:41.9160688167454, 0:05, 20:00 (text box)
- Availability:** ☒ (checkbox), 95 (text box), % (text box), MTTR (text box), 1:00, 0:05, 20:00 (text box)
- Failure Relates To:** SimulationTime (dropdown)
- Buttons:** OK, Cancel, Apply

If you select any of the provided distributions, you have to enter the parameters that this distribution requires. *Plant Simulation* will enter the resulting value for the **MTTR** into the text box **MTTR**, you cannot edit it.

The dialog box 'Models.MyPlant.MyMachine1' is used to configure a failure profile. It contains the following fields and options:

- Name:** Failure
- Failed:** ☐
- Active:** ☒
- Start:** Uniform (dropdown), 30:00, 35:00 (text box)
- Stop:** Const (dropdown), 0 (text box)
- Interval:** Negexp (dropdown), 19:00 (text box)
- Duration:** Normal (dropdown), 1:00, 15:00, 0:05, 0:10 (text box)
- Availability:** ☒
- Availability:** 100 % MTTR 0
- Failure Relates To:** SimulationTime (dropdown)
- Buttons:** OK, Cancel, Apply

8. If you want to select a distribution for the **Interval** and the **Duration**, clear the check box **Availability**. Then select a distribution for the time between the end of the last failure and the beginning of the next one, i.e., the failure Interval, from the drop-down list. Enter the values, which that distribution requires, into the text box. *Plant Simulation* shows these values above the list of distributions which you can select.

Select a distribution for the **Duration** of the failure. Enter the values, which that distribution requires, into the text box. *Plant Simulation* shows these values above the list of distributions which you can select.

The dialog box 'Models.MyPlant.MyMachine2' is used to configure a failure profile. It contains the following fields and options:

- Name:** MyFailureProfile
- Failed:** ☐
- Active:** ☒
- Start:** Uniform (dropdown), 30:00, 35:00 (text box)
- Stop:** Const (dropdown), 0 (text box)
- Interval:** Negexp (dropdown), 19:00 (text box)
- Duration:** Erlang (dropdown), 0:59.2782730020053, 0:41.9160688167454, 0:05, 20:00 (text box)
- Availability:** ☐
- Availability:** 95 % MTTR 1:00, 0:05, 20:00
- Failure Relates To:** SimulationTime (dropdown)
- Buttons:** OK, Cancel, Apply

9. Select the time to which the failures relate from the drop-down list:

- **Simulation Time**

Consumes the time you entered for the failure interval, regardless of the state the object is in. An example could be the electrical system of the plant, which may fail at any point in time. The **Simulation Time** is the time between the beginning of the simulation run (**Reset Simulation**, **Start Simulation**) and its end (**Stop Simulation**).

- **Operating Time**

Consumes the time you entered for the failure interval only if the object is operational. An example could be the coolant pump of the engine, which may fail any time during which the machine is on; the machine does not actually have to process parts. The **Operating Time** will be interrupted by **pauses** and **failures**.

- **Processing Time**

Consumes the time you entered for the failure interval while the object is processing. An example could be a saw blade, which can only break, when the machine actually saws materials.

For point-oriented objects the **Processing Time** is the time during which a *part* is located on a material flow object and is being processed by it.

For conveying objects, such as the *Conveyor*, the **Processing Time** is the time during which its **speed** is not 0. It can also be *working*, when the conveyor is moving and does not transport a part.

For processing time based failures of a *ParallelStation*  the simulated MTBF is reduced, when the number of parallel processing operations of parts on the *ParallelStation* increases. This means that the more parts are processed simultaneously, the sooner the failure occurs, and the smaller the availability becomes. This then prevents the **Availability** you entered into the dialog from being reached.

Material Flow Objects	Transporter, Exporter, Worker
Failure Relates To SimulationTime SimulationTime OperatingTime ProcessingTime	Failure Relates To SimulationTime SimulationTime OperatingTime

10. Click **OK** to add this *failure profile* to the list of *failure profiles*. If you want to edit a *failure profile*, double-click it in the list or click **Edit**.

11. Repeat this procedure for any additional *failure profiles* you want to define.

Go to [Change Failure Settings During the Simulation Run](#)

Go to [Random Numbers and Their Statistical Distribution](#)

Compare [Model Random Processes](#)

Compare [Specify Times in Object Dialogs](#)

Back to [Model Failures](#)

Video on YouTube

<https://youtu.be/PQhEriOzVzU?si=8y6fctOOliWSL2gp&t=118>

Change Failure Settings During the Simulation Run

In real-life machines fail more often as they grow older. This is why you will want to reduce the **Availability** of the machine in the simulation model during the simulation run after a certain time has passed.

- To change the **Availability** and the **MTTR** of our station *MyStation* via *SimTalk*, enter the following instructions into a *Method*. In our example we called this method *setFailure*.

Note

As our machine *MyStation* only has a single *failure profile*, we can type in the instructions in the simplified notation as below. *Plant Simulation* then always uses the first *failure profile*. If the machine had several *failure profiles*, we would have to address the attributes via the respective *failure profile*, *MyStation.Failures.MyFailureProfile1.Availability := 80* for example.

The source code looks like this:

```
// Reduces the availability and the MTTR during the simulation run.  
// This method is called by the init method via a method call after 12  
hours.  
print EventController.simTime," Simulated availability in the morning:  
",round(100 * (1 - MyStation.StatFailPortion),2)," %"  
MyStation.initStat  
// reduce the availability  
MyStation.Availability := 80  
MyStation.MTTR := str_to_time("10:00")  
MyStation.FailureActive := false  
MyStation.FailureActive := true // activate failures
```

- To start reducing the **Availability** of the machine after 12 hours have been simulated, enter the following instructions into the *init method* in the model:

```
&setFailure.executeIn(str_to_time("12:0:0"))
```

- To reset the **Availability** of the machine at the start of the next simulation run to 100 percent, enter the following instruction into the *reset method* in the model.

```
MyStation.Availability := 100
```

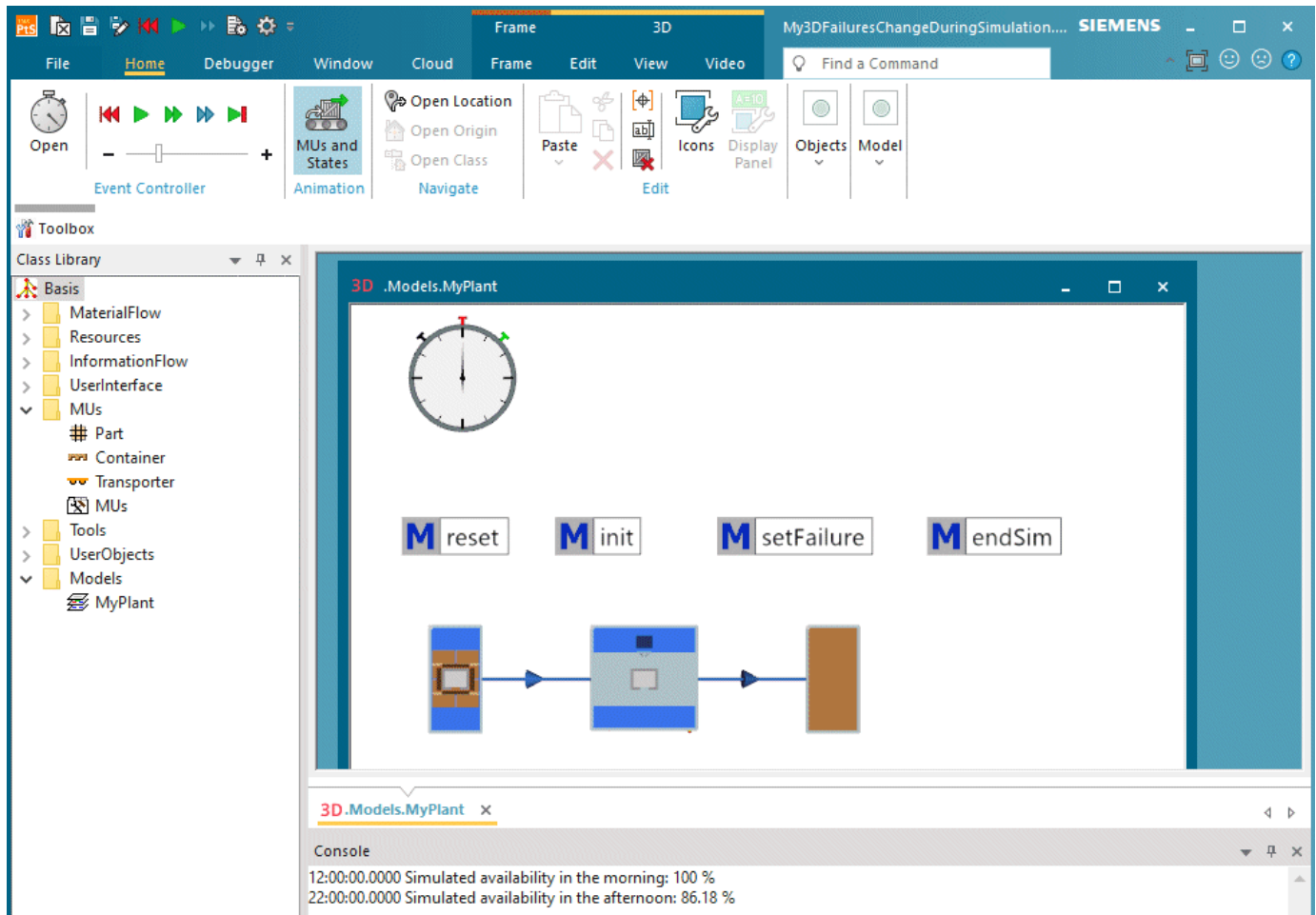
- To show the simulated **Availability** at the end of the simulation run in the *Console*, enter the following instruction into the *endSim* method in the model.

```
print EventController.simTime," Simulated availability in the afternoon: ", round(100 * (1 - MyStation.statFailPortion),2)," %"
```

- Type in 1:00:00:00 to simulate a day
- Then, run the simulation and watch what *Plant Simulation* shows in the *Console*.

For the first 12 hours of the simulation run the **Availability** of the object *MyStation* was 100 percent. Then we set the **Availability** to 80 percent and the **MTTR** to 10 minutes and deactivated and reactivated *failures*, to make sure that the next failure is computed with a complete set of valid parameters.

After an additional 10 hours of simulating, compare the end time of the *EventController*, the **Availability** of the machine *MyStation* has decreased to 86.18 percent.



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

Back to [Configure Failures](#)

Compare [Model Random Processes](#)

Compare [Specify Times in Object Dialogs](#)

Back to [Model Failures](#)

Buffering Parts within the Production Line

Buffer Parts within the Production Line

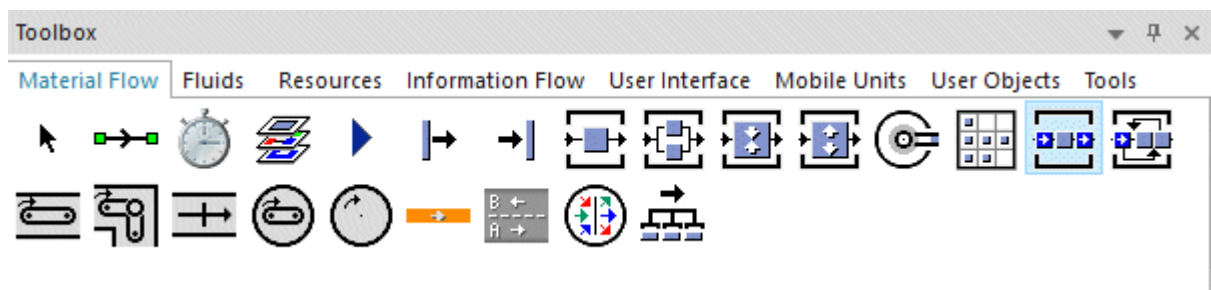
The **Buffer** , which you place between two components or stations of your plant, serves two purposes:

- It temporarily holds parts, when one of the components following it in the sequence of stations fail.
- It moves parts on, when the preceding components stop working, preventing the production process from grinding to a halt.

Dimensioning a *Buffer* with a large enough **capacity** for covering all failures leads to a complete decoupling of both components of your plant.

The *Buffer* not only tides over failure times but also serves as a compensating station for fluctuating transport and operating times, which lead to queues forming in front of a machine or a component. But even then it cannot always prevent the material flow from being interrupted or coming to a halt.

You can insert the *Buffer* into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **MaterialFlow** in the *Toolbox*.



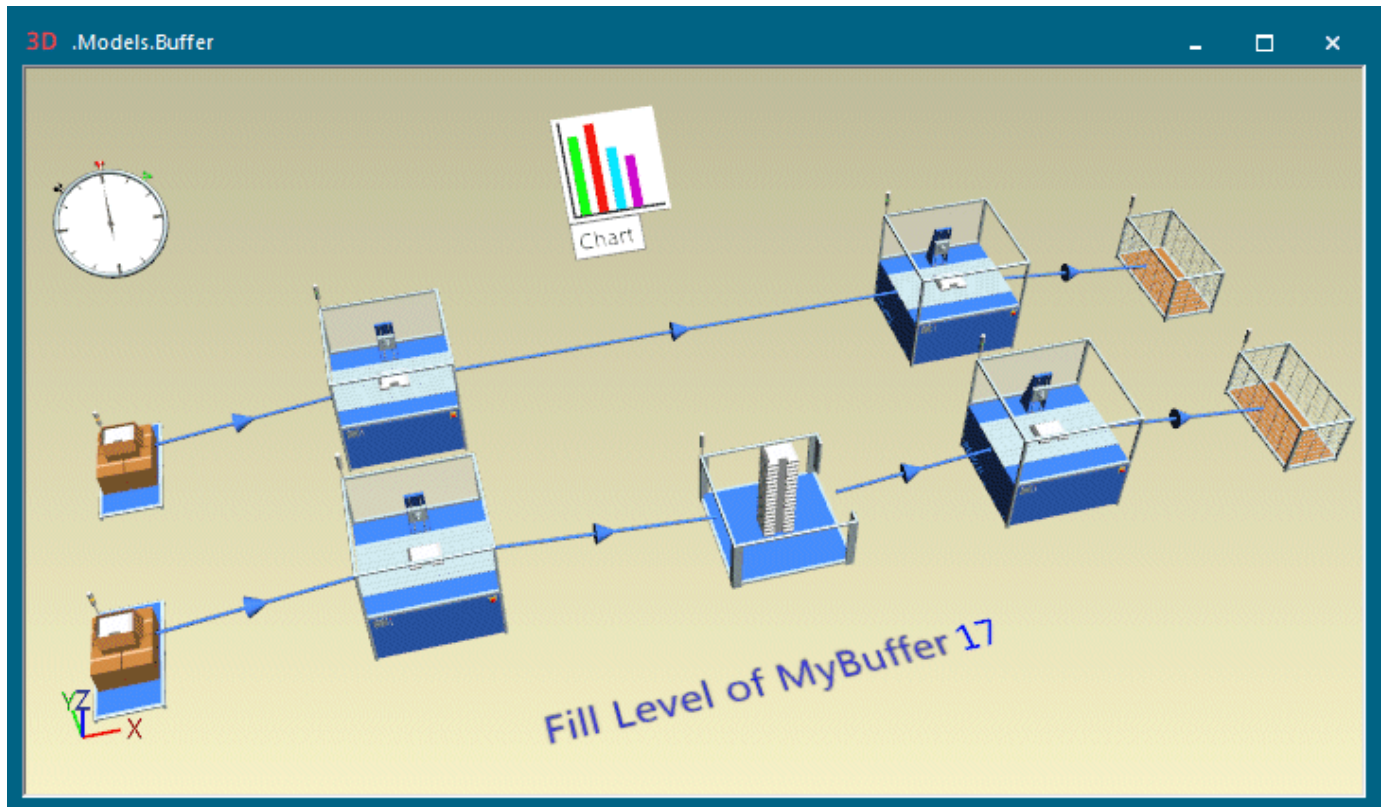
Below we demonstrate how to:

- [Use a Buffer between Processing Stations](#)
- [Check the Fill Level of the Buffers in the Plant](#)

Back to [Model the Flow of Materials, Basics](#)

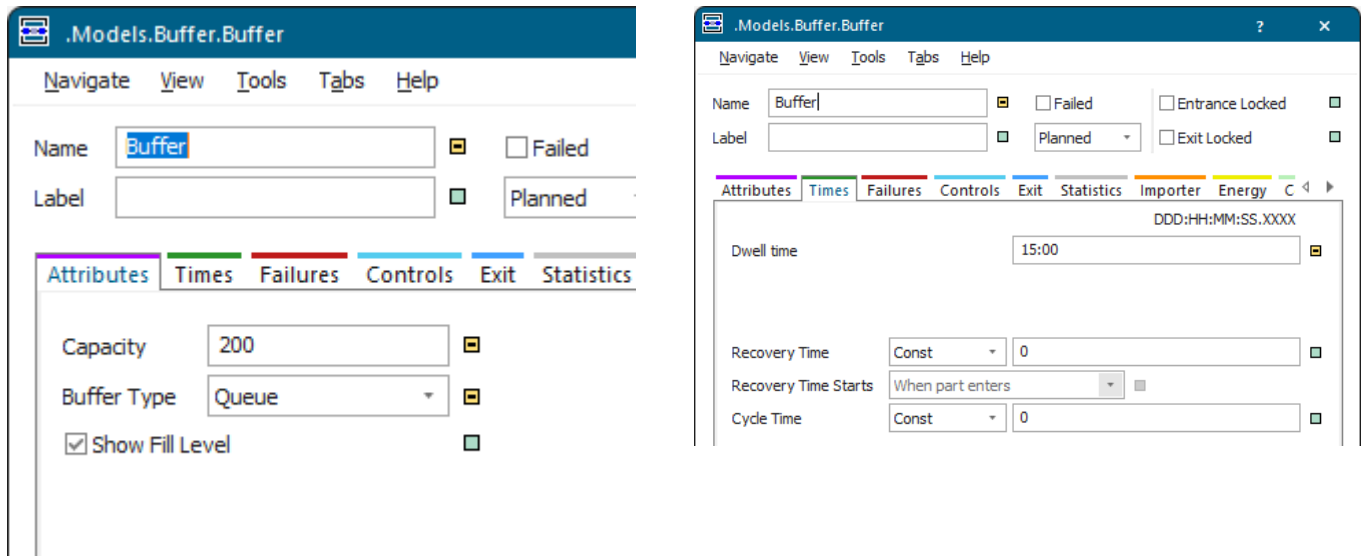
Use a Buffer between Processing Stations

In our sample model consisting of a simple production line with a *source*, a *processing station*, a *test station*, and a *drain*, the *processing station* is blocked now and then because the *test station* cannot test the parts fast enough.



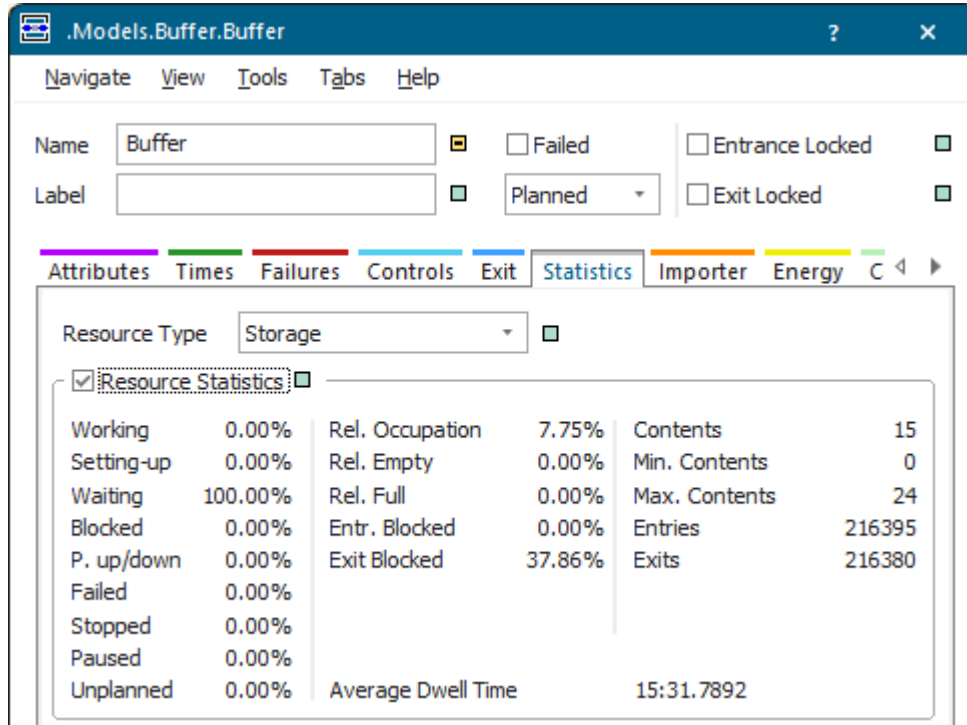
To remedy the situation, we inserted a **Buffer**  between the *processing station* and the *test station*. The *Buffer* temporarily holds the parts before moving them on to the *test station*.

As compared to the default settings we changed the **capacity** of the *Buffer* from **4** to **200** and entered a processing time of 15 minutes. We left the remaining settings unchanged.

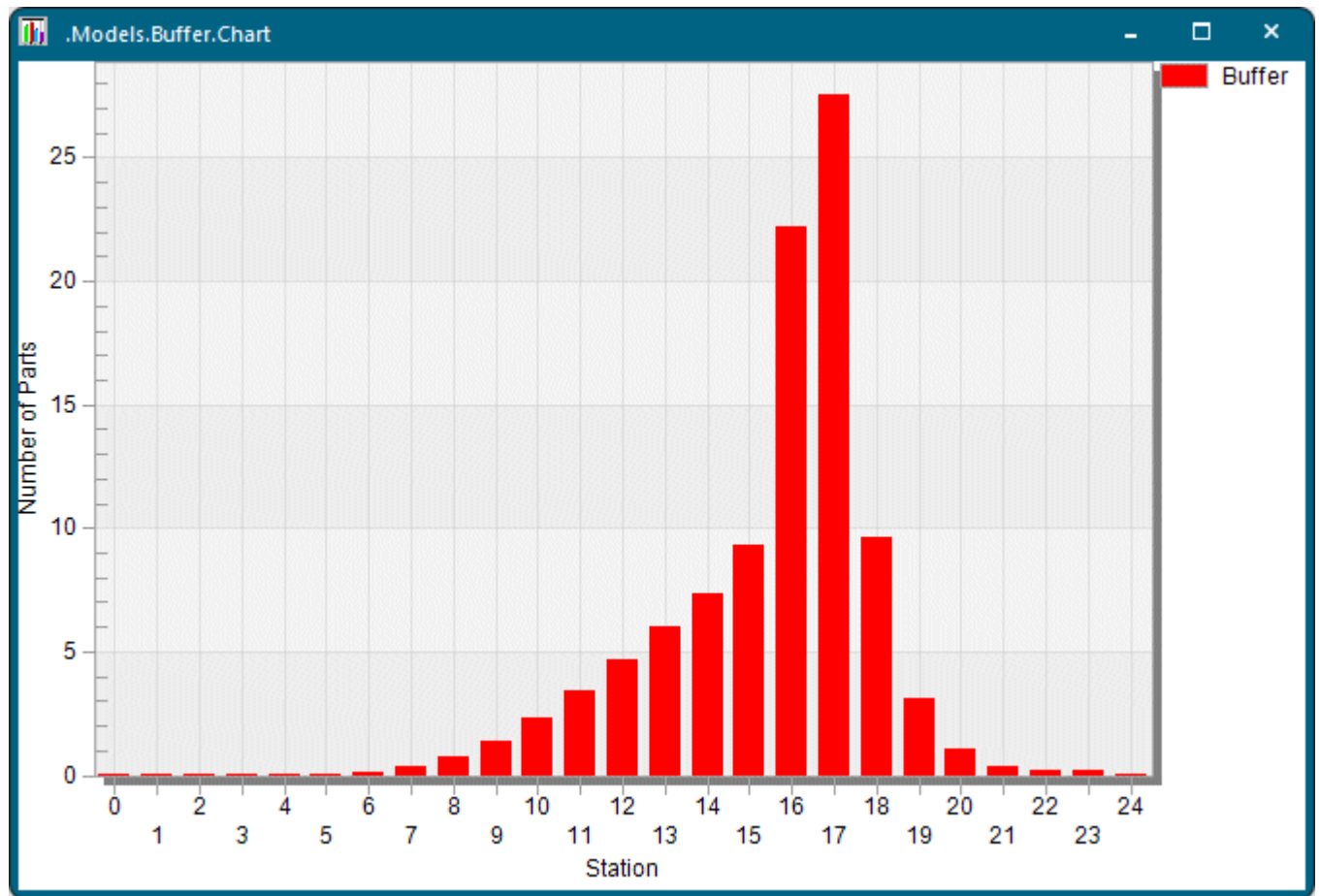


When you now run the simulation, you will notice that the *processing station* will not be blocked because the *Buffer* temporarily holds a maximum of 200 parts for 15 minutes each before the *testing station* can test them.

The tab **Statistics** shows the relative occupancy, the percentage of the time it was relatively empty or relatively full, maximum and minimum contents, and the number of entries and exits.



You can also use a *Chart* to show how the number of parts in the *Buffer* develops over time.



Go to [Check the Fill Level of the Buffers in the Plant](#)

Back to [Buffer Parts within the Production Line](#)

Checking the Fill Level of the Buffers

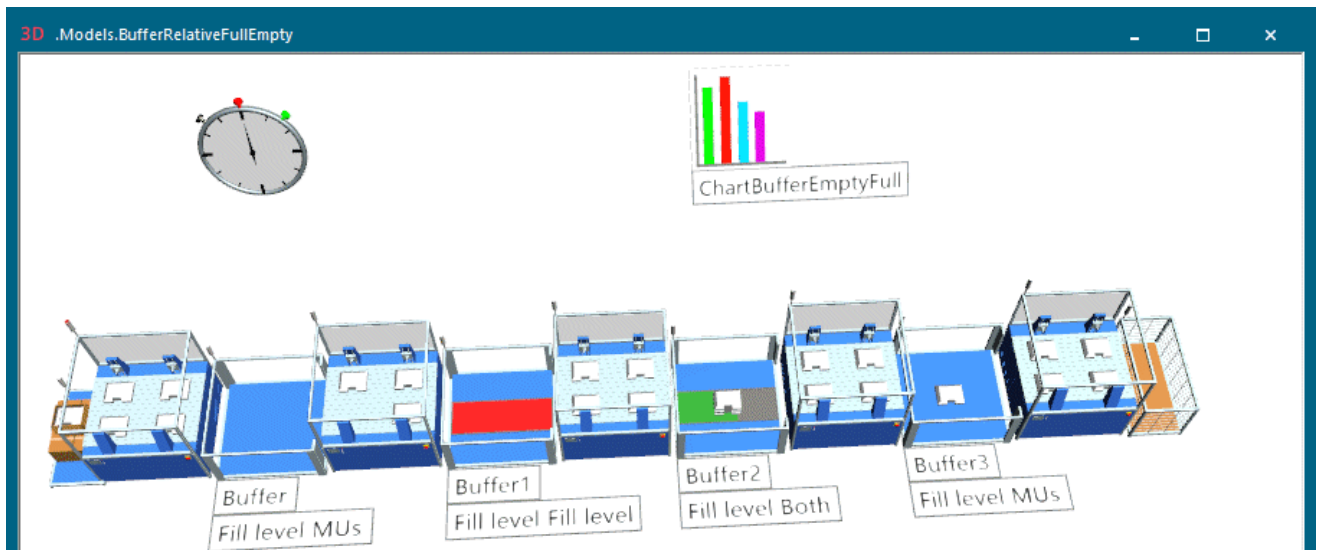
Check the Fill Level of the Buffers in the Plant

At times you may want to show the relative fill level of the *Buffer* in the plant.

Proceed as follows :

- To display the fill level of the **Buffer**  in 3D, select **Fill level** from the drop-down list **Show Contents As**.
- To view more exact percentages, click the tab **Statistics**, and check the following values:
 - **Relatively empty** shows the portion of the statistics collection period during which the object was empty in relation to the time during which the object was available.
 - **Relatively full** shows the portion of the statistics collection period during which all temporary storage places of the *Buffer* were occupied in relation to the time during which the object was available.

The finished simulation model for demonstrating these features looks like this:



To create the sample model, you will:

- [Configure the Processing Stations](#)
- [Configure the Buffers](#)
- [Configure the Chart for Showing the Full and Empty Portions](#)
- [Show the Fill Level of the Buffers](#)

[Back to Use a Buffer between Processing Stations](#)

[Back to Buffer Parts within the Production Line](#)

Configure the Processing Stations

Insert and configure the processing stations, which process the parts for a certain time before passing them on to their successors in the succession of stations into the factory.

Proceed as follows:

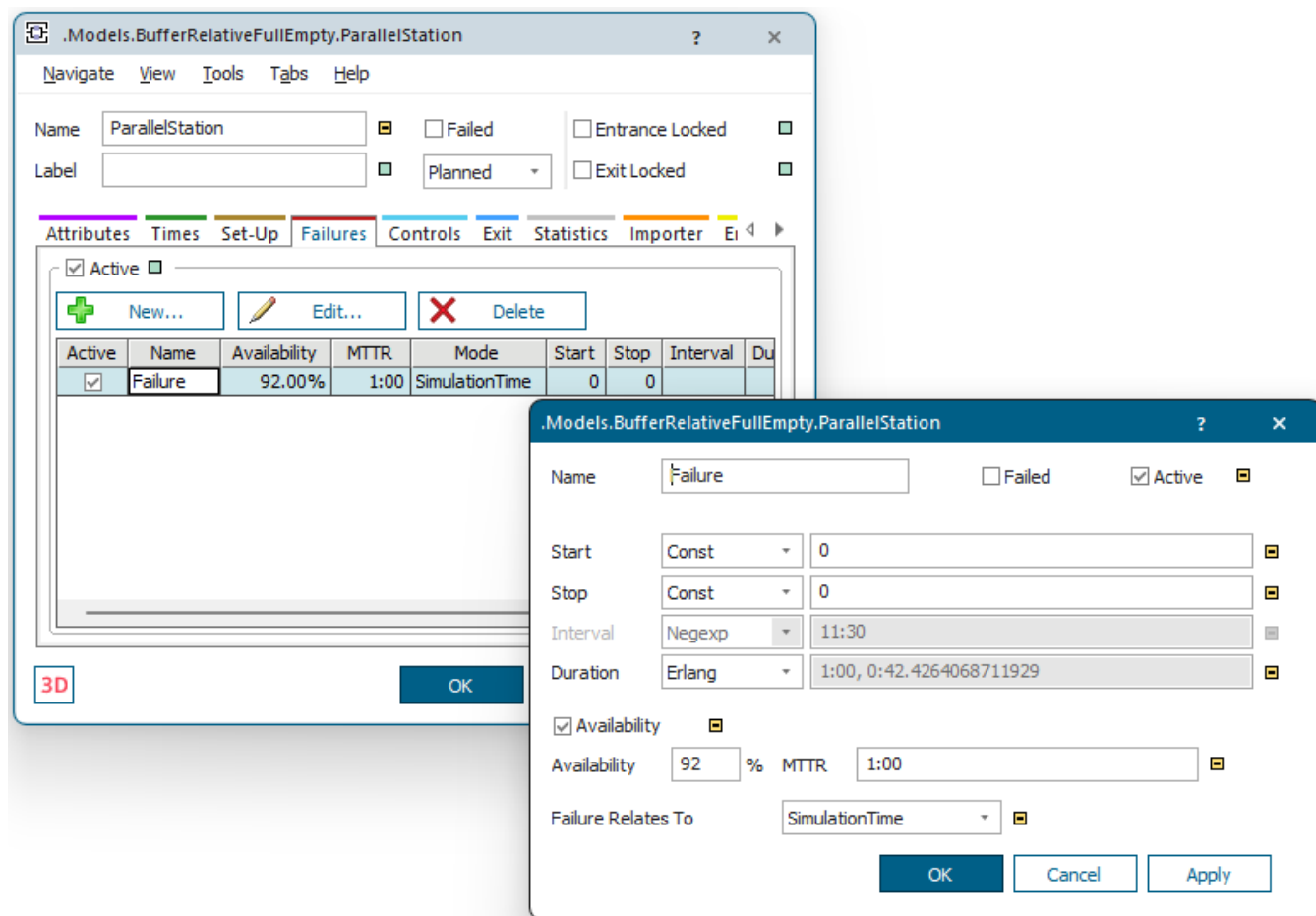
- We inserted five stations of type **ParallelStation**  in our sample model.
- Enter the **Capacity** of the processing stations. We used the default setting, namely 2 as the **X-Dimension** and as the **Y-Dimension** respectively. This results in 4 processing places on each station.

The screenshot shows the 'ParallelStation' configuration window with the 'Attributes' tab selected. The window title is '.Models.BufferRelativeFullEmpty.ParallelStation'. The 'Name' field is 'ParallelStation' and the 'Label' field is empty. There are checkboxes for 'Failed', 'Entrance Locked', and 'Exit Locked', all of which are unchecked. A 'Planned' dropdown menu is visible. Below the tabs, the 'X-Dimension' is set to 2 and the 'Y-Dimension' is set to 2. There is also a checkbox for 'Start Processing When Full' which is unchecked.

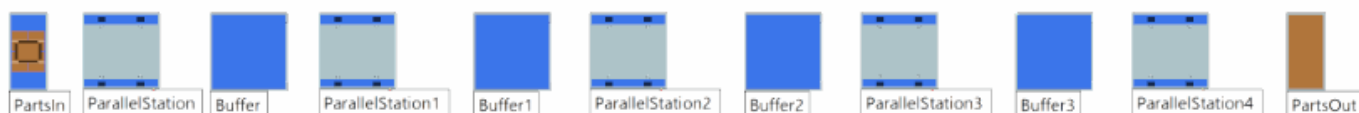
- Enter a **Processing Time**. We decided to use a **normally distributed time** with the values shown below.

The screenshot shows the 'ParallelStation' configuration window with the 'Times' tab selected. The window title is '.Models.BufferRelativeFullEmpty.ParallelStation'. The 'Name' field is 'ParallelStation' and the 'Label' field is empty. There are checkboxes for 'Failed', 'Entrance Locked', and 'Exit Locked', all of which are unchecked. A 'Planned' dropdown menu is visible. Below the tabs, the 'Processing Time' is set to 'Normal' with a distribution of '1:00, 0:10, 0:30, 1:40'. There is a checkbox for 'Automatic Processing' which is checked. The 'Set-up Time' is set to 'Const' with a value of 0. The 'Recovery Time' is set to 'Const' with a value of 0. The 'Recovery Time Starts' is set to 'When part enters'. The 'Cycle Time' is set to 'Const' with a value of 0.

- Enter settings for **Failures** of the processing station. We decided to use an **Availability** of 92 percent.



We used the same settings for the other processing stations as well.

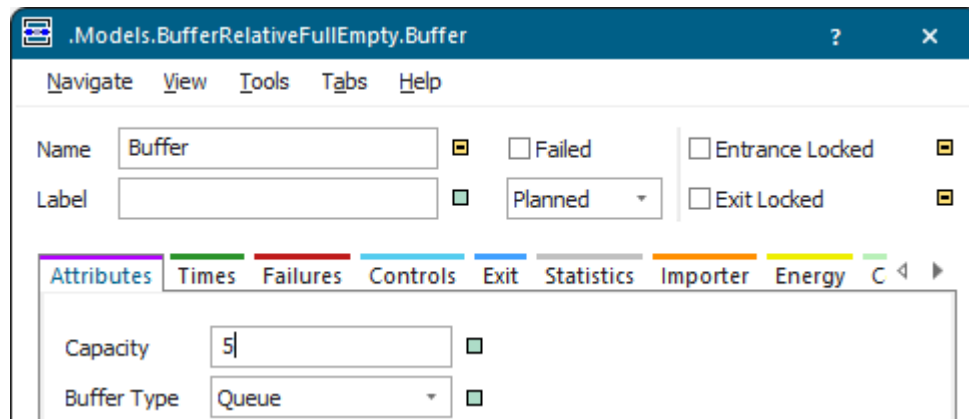


Back to [Check the Fill Level of the Buffers in the Plant](#)

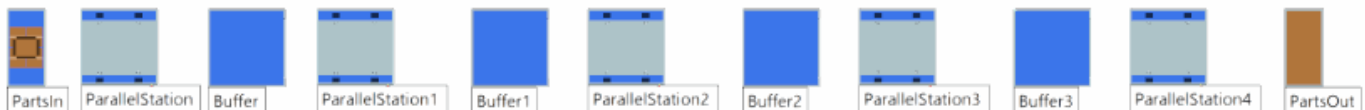
Configure the Buffers

Insert and configure the *Buffers*, which temporarily hold the parts.

We inserted four *Buffers* into our sample model and used the settings below for all of them.



Insert and configure the rest of the stations so that they match the screenshot below.



Then, run the simulation and check the tab **Statistics** of the individual *Buffers*, specifically the values for relatively empty and relative full in the middle column of the tab.

Models.BufferRelativeFullEmpty.Buffer

?

×

NavigateViewToolsTabsHelp

NameBuffer

☐ Failed☐ Entrance Locked

Label

☐ Planned☐ Exit Locked

AttributesTimesFailuresControlsExitStatisticsImporterEnergyC

Resource TypeStorage

☒ Resource Statistics

Working	0.00%	Rel. Occupation	49.55%	Contents	5
Setting-up	0.00%	Rel. Empty	38.83%	Min. Contents	0
Waiting	61.95%	Rel. Full	38.05%	Max. Contents	5
Blocked	38.05%	Entr. Blocked	14.76%	Entries	103
P. up/down	0.00%	Exit Blocked	61.17%	Exits	98
Failed	0.00%				
Stopped	0.00%				
Paused	0.00%				
Unplanned	0.00%	Average Dwell Time	43.7015		

Models.BufferRelativeFullEmpty.Buffer3

?

×

NavigateViewToolsTabsHelp

NameBuffer3

☐ Failed☐ Entrance Locked

Label

☐ Planned☐ Exit Locked

AttributesTimesFailuresControlsExitStatisticsImporterEnergyC

Resource TypeStorage

☒ Resource Statistics

Working	0.00%	Rel. Occupation	36.23%	Contents	5
Setting-up	0.00%	Rel. Empty	52.56%	Min. Contents	0
Waiting	77.28%	Rel. Full	22.72%	Max. Contents	5
Blocked	22.72%	Entr. Blocked	12.21%	Entries	81
P. up/down	0.00%	Exit Blocked	47.44%	Exits	76
Failed	0.00%				
Stopped	0.00%				
Paused	0.00%				
Unplanned	0.00%	Average Dwell Time	40.6318		

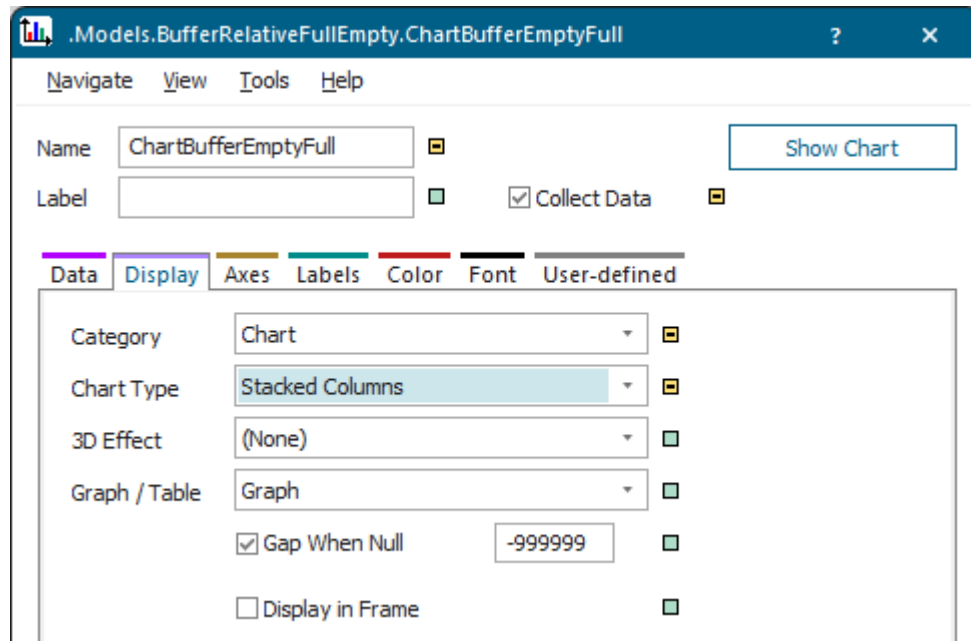
Back to [Check the Fill Level of the Buffers in the Plant](#)

Configure the Chart for Showing the Full and Empty Portions

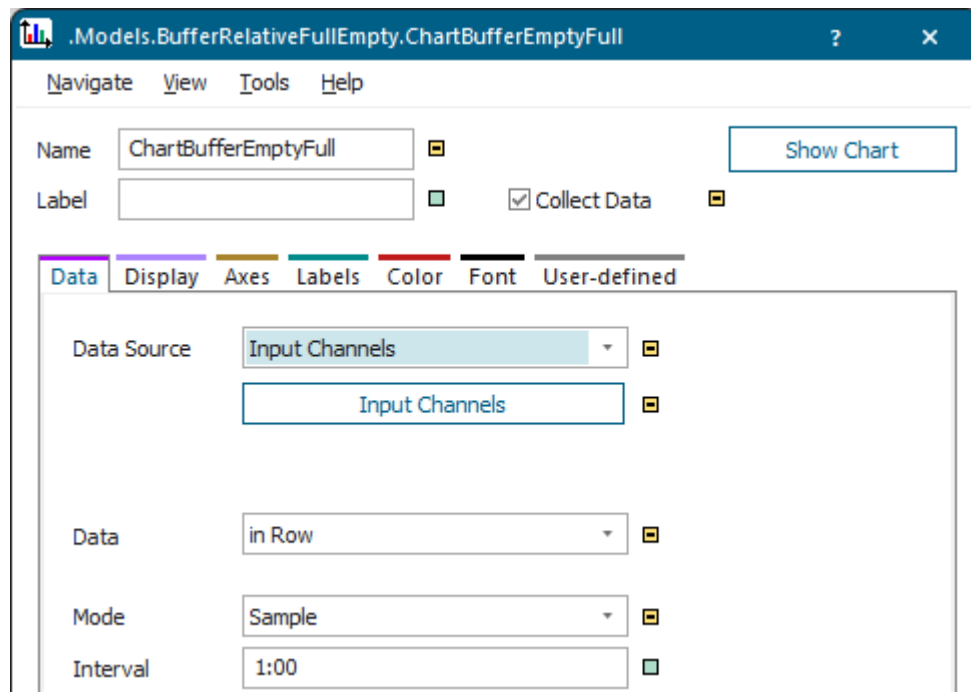
In addition to viewing the percentage values for **relatively empty** and **relative full** on the tab **Statistics**, you can also display these values in a *Chart*.

Proceed as follows:

- Insert a *Chart* into the simulation model.
- Click the tab **Display** and select the **Category > Chart**.



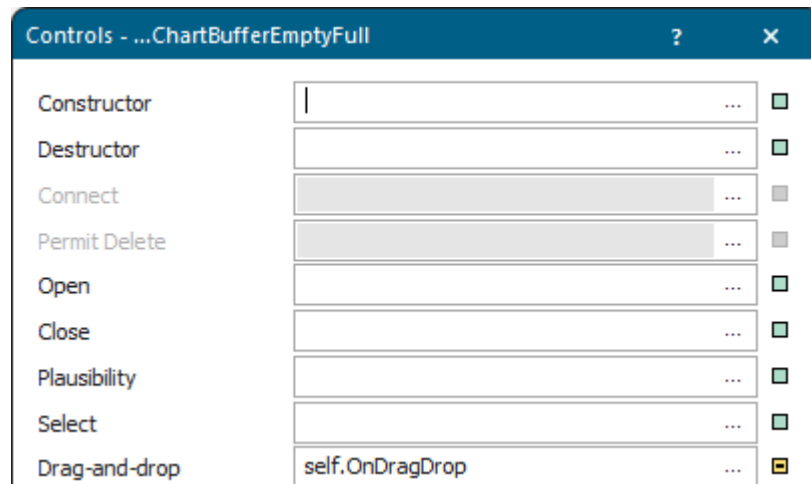
- Click the tab **Data** and select the **Data Source > Input Channels**.
- Deactivate inheritance of the input channels table and click **Input Channels**.



- Enter the labels of the values you want to display into the table. We entered Empty, Partially full, and Full into the cells of the row index.

	string 0	string 1	string 2	string 3	string 4
string					
1	Empty				
2	Partially full				
3	Full				
4					

- As we want to drag the *Buffers* onto the *Chart* and drop them there to display the values, we need to create a **drag-and-drop control** for the *Chart*:
 - Select **Tools > Edit Controls**, click in the text box next to **drag-and-drop**, and press the **F4** key. This creates the **drag-and-drop** control as a *user-defined attribute* of data type *method*.



- Copy the following source code and paste it into the method that opens:

```
param draggedObjects: object[]
var obj: object
var numBuffers: integer
var MyTab : table := @.InputChannels // inheritance of the table
'InputChannels' is already deactivated
MyTab.delete({1,0}..{*,*})

for var i := 1 to draggedObjects.dim
  obj := draggedObjects[i]

  switch obj.internalClassType
  case "Buffer", "PlaceBuffer", "Sorter"

    numBuffers := self.~.InputChannels.xDim +1

    MyTab[numBuffers,0] := obj.Name
    MyTab[numBuffers,1] := obj.Name + ".statRelativeEmptyPortion"
    MyTab[numBuffers,2] := "1 - " + obj.Name +
```



```

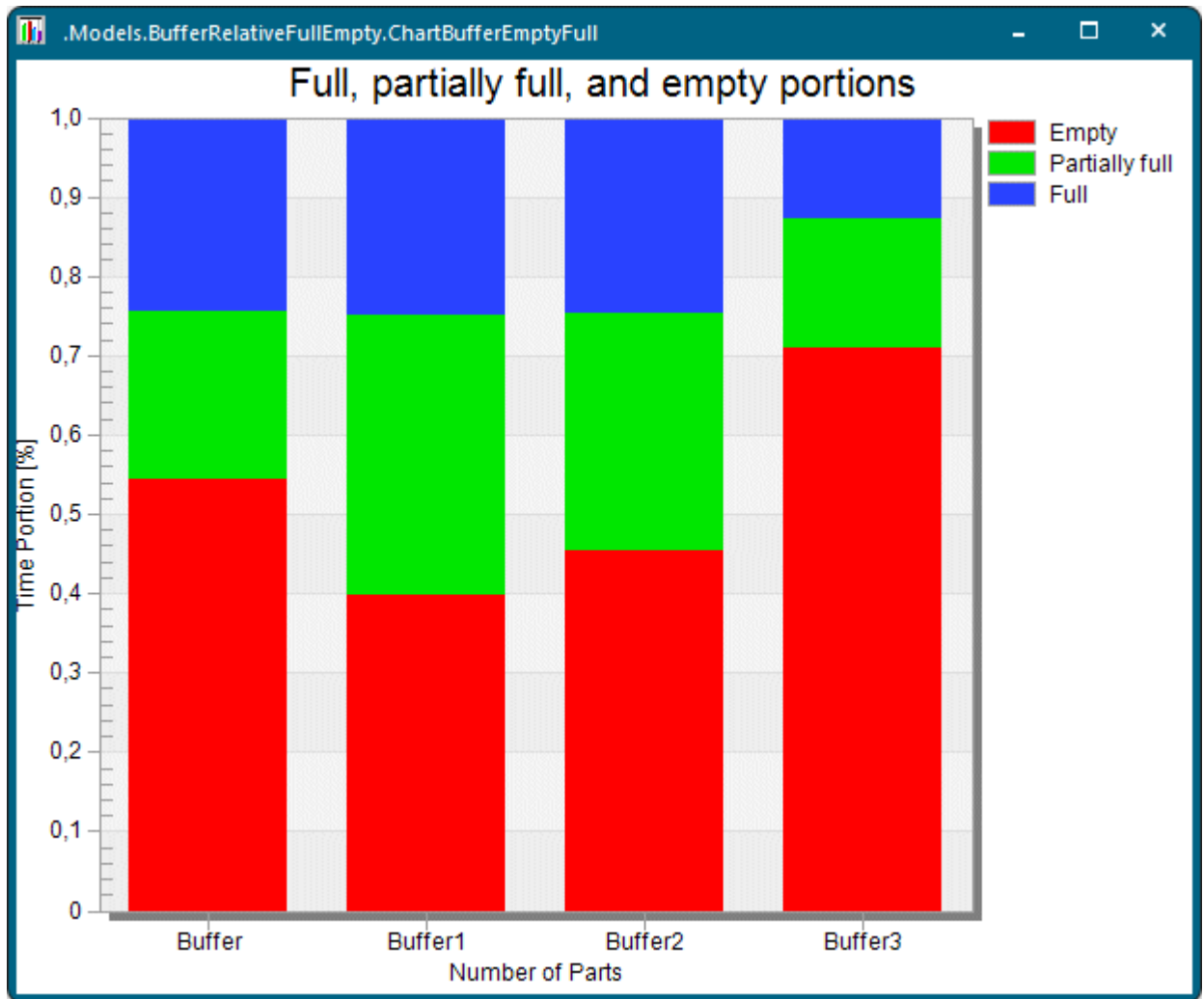
".statRelativeEmptyPortion - " + obj.Name + ".statRelativeFullPortion"
  MyTab[numBuffers,3] := obj.Name + ".statRelativeFullPortion"

  end
next

print "Number of detected buffers: ", numBuffers
@.InputChannels := MyTab // makes the chart apply the new settings
@.Active := true

```

- Drag a marquee over all objects in the model, drag them onto the *Chart*, and drop them there. The *Chart* automatically knows for which of the objects it can display values and then displays them.

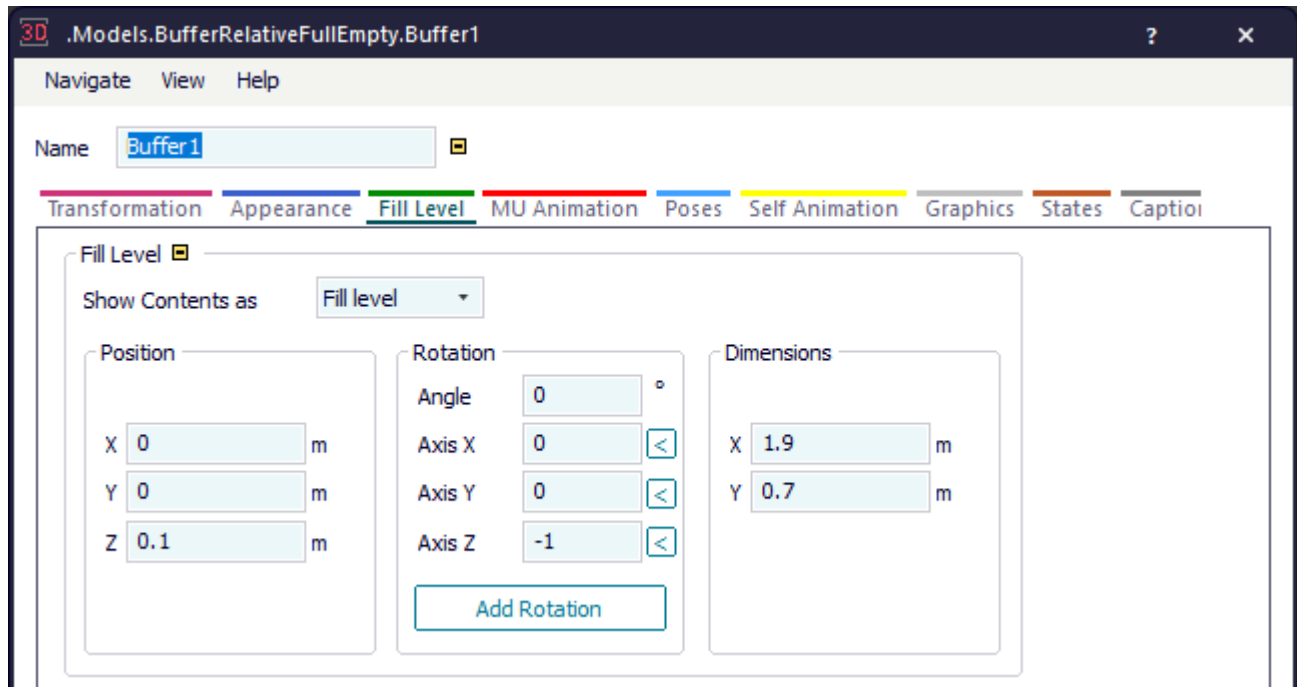


Back to [Check the Fill Level of the Buffers in the Plant](#)

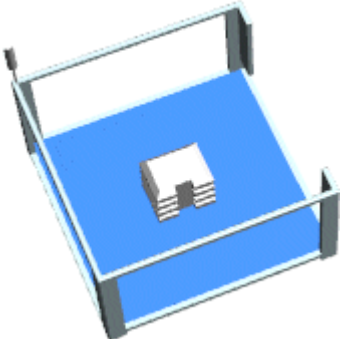
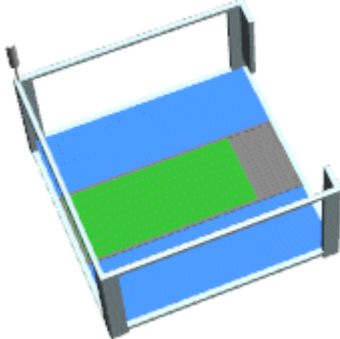
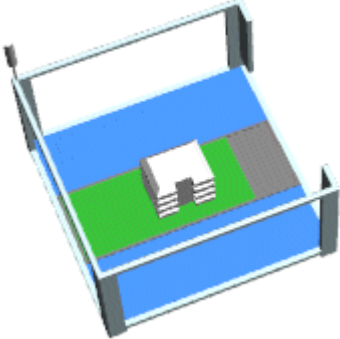
Show the Fill Level of the Buffers

You can show the **Fill Level** of the *Buffers* in different ways.

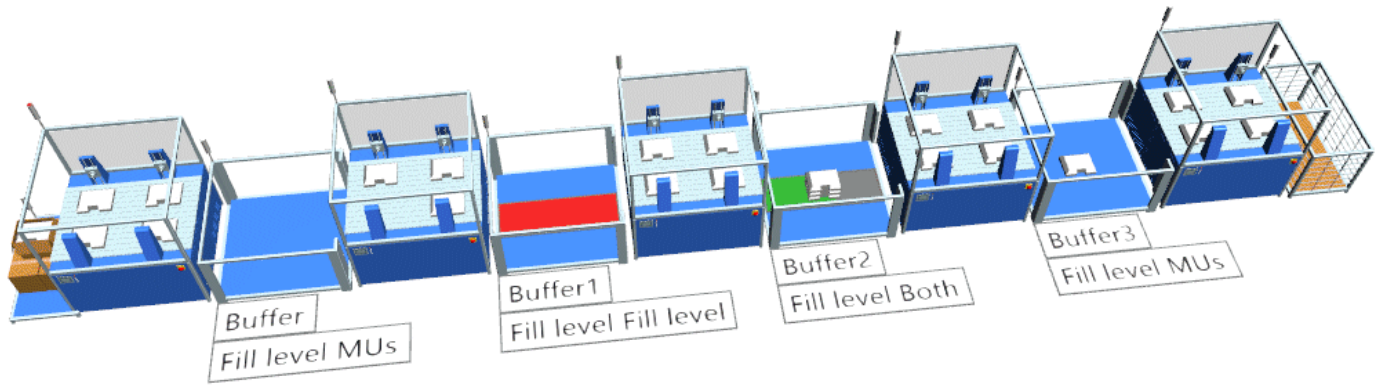
To display the fill level of the *Buffer*, select **Fill Level** from the drop-down list **Show Contents As** on the **Tab Fill Level** of the dialog **Edit 3D Properties**. You can also define settings for the **Position**, the **Rotation**, and the **Dimension** of the fill level indicator.



The fill level bar just shows a quantitative portion of the *MUs* located in the *Buffer*, not exact numbers.

Item	Description	Looks like this
MUs	Shows the <i>MUs</i> stacked in the <i>Buffer</i>. Is the setting in previous versions of <i>Plant Simulation</i>.	
Fill level	Shows the relative fill level of the <i>Buffer</i> on its picture. Gray denotes the background of the fill level indicator. Red denotes a fill level of more than 90%. Orange denotes a fill level of less than 10%. Green denotes a fill level between the above settings.	
Both	Shows the <i>MUs</i> stacked on top of the fill level indicator.	

The different settings look like this in our model.



[Back to Configure the Processing Stations](#)

[Back to Configure the Buffers](#)

[Back to Configure the Chart for Showing the Full and Empty Portions](#)

[Back to Check the Fill Level of the Buffers in the Plant](#)

Placing Parts into Stock and Removing Them

Place Parts into Stock and Remove Parts from It

When you model your plant you will, at one point, also face the task of placing parts into stock and removing them from stock. To model a warehouse, you will use the object *Store*. The parts remain in the *Store* until you remove them with a *Method*.

The **Store**  accepts parts as long as storage places are available. The incoming *part* triggers a sensor, when it enters the *Store*. The sensor then calls an **Entrance Control**, i.e., a *Method* object, that determines the storage place onto which the *Store* deposits the *part*. The **Entrance Control** can, for example, update the inventory list or execute any other action which you program. If you do not use an **Entrance Control**, the *Store* deposits the *part* into the first free storage place in the net of coordinates of the storage places.

In our sample model the material flow is controlled by the method *placeInStock*, which we use as the **Entrance Control** of the warehouse, and by the method *removeFromStock*, which we use as the **Exit Control** of the station *RetrieveFromStore*.

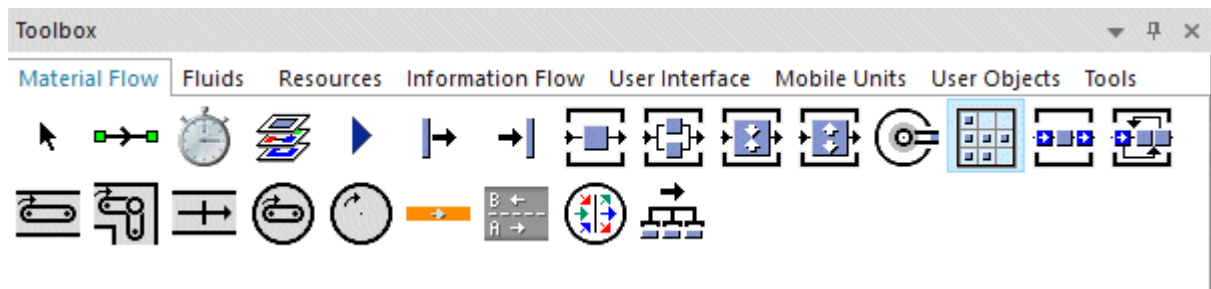
As parts occasionally have to be reworked, we want to change the original order of parts, which are placed into stock, and we then want to re-establish the original order when we remove them from stock. We do this in the station *MixOrders* located directly before our *Store*.

We have to set the size of the *Store* according to the degree of change of the sequence to prevent it from overflowing. To accomplish this, two events are essential: First an additional part arrives at the *Store* and second a part is removed from the *Store*. When the *Store* is full, it does not accept any additional parts. We therefore have to increase the size of the *Store* in the **Entrance Control** of the *Store* is full. Each part that is placed into stock and that is removed from stock must be registered on the

data table *InventoryTable*. To manage the inventory and to remove parts from stock, we program the control *AttemptToRemoveNextPart*, which is called by the methods *placeInStock* and *removeFromStock*. The method *AttemptToRemoveNextPart* checks whether the next removing action is possible so that the original order of the parts can be re-established so that the production process can be maintained without a shortage of parts.

In the *reset* method we delete the contents of the *InventoryTable* and reset the counter of the variable named *NextNumber* to 1.

You can insert the *Store* into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **MaterialFlow** in the *Toolbox*.

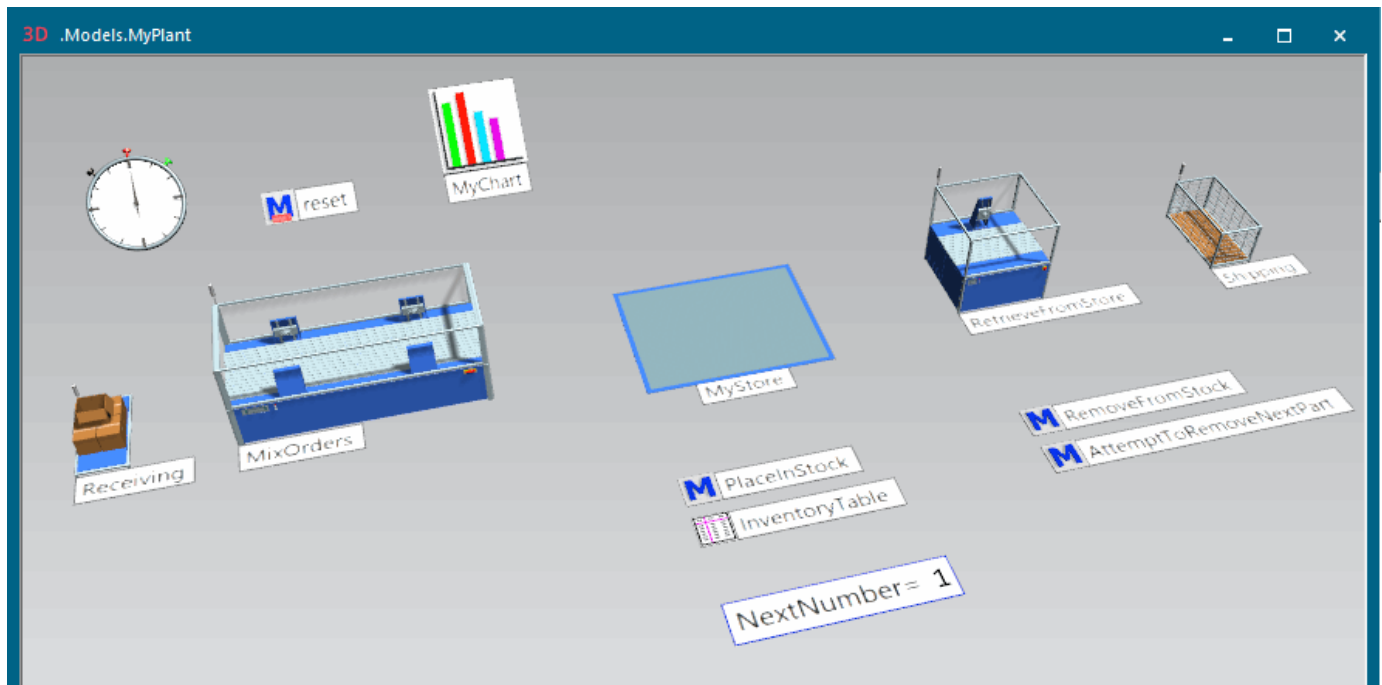


First we insert the material flow objects that we need:

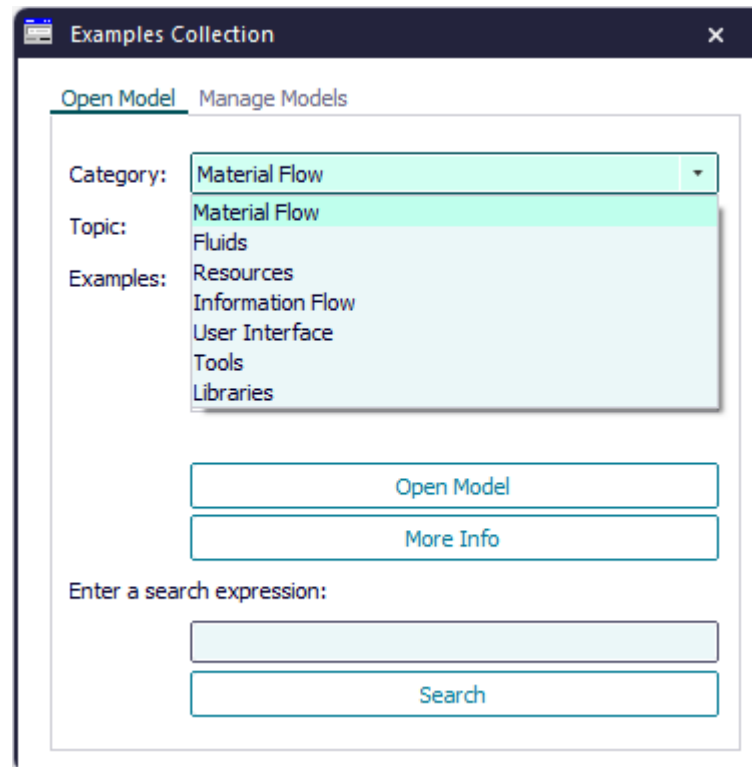
- The *Source* named *Receiving* creates the parts.
- The *ParallelStation* named *MixOrders* mixes the original production sequence.
- The *Store* places parts in stock and removes them from it.
- The *Station* named *RetrieveFromStore* retrieves the required parts from the *Store*.
- The *Drain* named *Shipping* removes the parts from the plant.

Then we insert the *Methods* and the visualization objects *Chart* and *Variable*. We finally connect the objects *Receiving*, *MixOrders*, and *MyStore* and *RetrieveFromStore* and *Shipping* with *Connectors*. We do not connect *MyStore* and *RetrieveFromStore* as retrieving parts is controlled by *Methods*.

The finished simulation model looks like this:



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



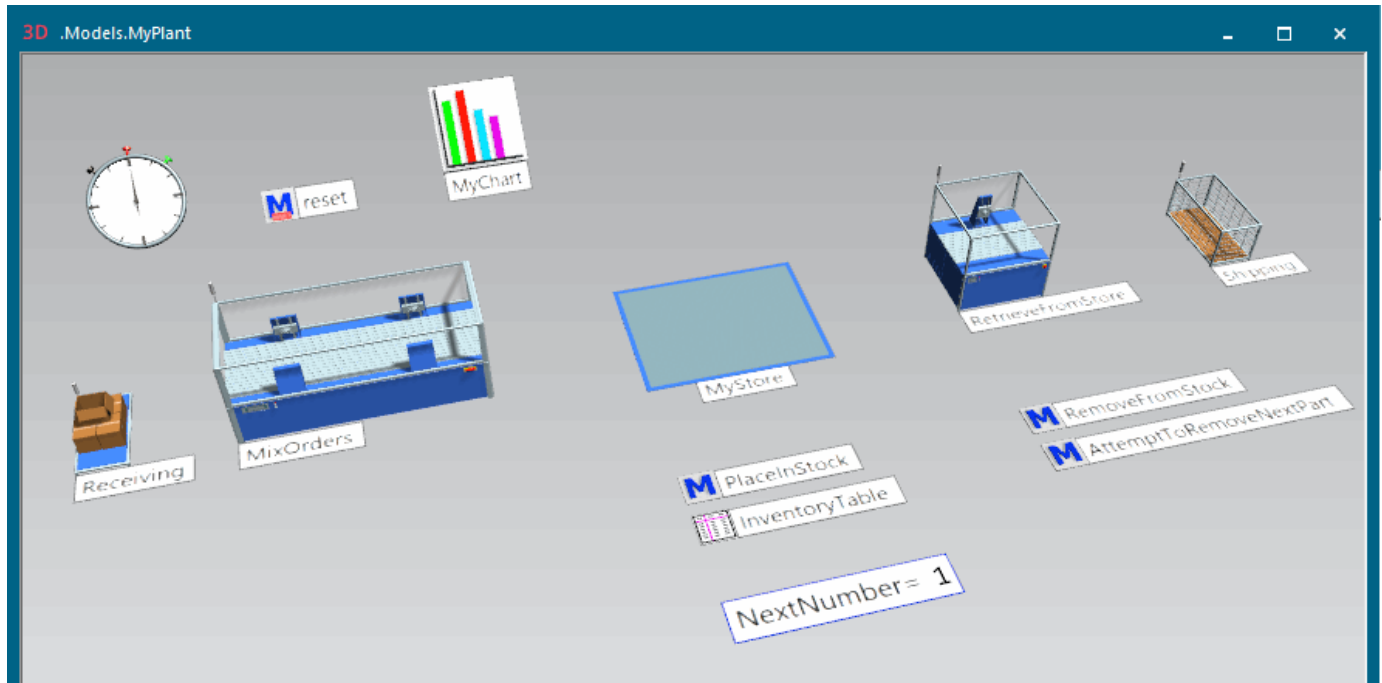
Compare **Stack Parts in the Store**

Back to [Model the Flow of Materials, Basics](#)

Configure the Stations

To create our simple warehouse model, we have to set the stations up.

Proceed as follows:



- To make the *Source* produce a part every minute, enter 1:00 as the **Interval**. To stop producing parts after a day, enter 1:00:00:00 for **Stop**.

.Models.MyPlant.Receiving

Navigate View Tools Help

Name: Receiving ☐ Failed

Label: ☒ Planned ☐ Exit Locked

Attributes Failures Controls Exit Statistics Importer User-defined

Operating Mode: ☒ Blocking ☐

Time of Creation: Interval Adjustable ☐ Amount: -1 ☐
DDD:HH:MM:SS.XXXX

Interval: Const 1:00 ☐

Start: Const 0 ☐

Stop: Const 1:00:00:00 ☐

MU Selection: Constant ☐

MU: *.MUs.Part ☐

3D ☒

OK Cancel Apply

- As parts occasionally have to be reworked, we want to mix up the original order of parts to be placed into stock, and we then want to re-establish the original order which is required for our production process when removing the parts from stock.

To set the size of the station *MixOrders*, which is a *ParallelStation*, according to the degree of change of the sequence of the arriving parts, enter an **X-Dimension** of 200 and an **Y-Dimension** of 20.

.Models.MyPlant.MixOrders

Navigate View Tools Tabs Help

Name: MixOrders ☐ Failed ☐ Entrance Locked ☒

Label: ☒ Planned ☐ Exit Locked ☒

Attributes Times Set-Up Failures Controls Exit Statistics Importer Ei

X-Dimension: 200 ☐ ☐ Start Processing When Full ☒

Y-Dimension: 20 ☐

OK Cancel Apply

To mix up the order of the parts that the *Source* produces, we need a random processing time. We selected the **Uniform** distribution and entered the parameters 0, 30:00 in our example. The two values denote the bounds of the **Uniform** distribution.

.Models.MyPlant.MixOrders

Navigate View Tools Tabs Help

Name: MixOrders ☐ Failed ☐ Entrance Locked ☒

Label: ☒ Planned ☐ Exit Locked ☒

Attributes Times Set-Up Failures Controls Exit Statistics Importer Ei

Lower Bound, Upper Bound

Processing Time: Uniform 0, 30:00 ☐

☒ Automatic Processing ☐

Set-up Time: Const 0 ☒

Recovery Time: Const 0 ☒

Recovery Time Starts: When part enters ☐

Cycle Time: Const 0 ☒

- To set the size of the *Store*, enter its **Dimensions** in the coordinate net of the storage places. We entered 4 and 8.

.Models.MyPlant.MyStore

Navigate View Tools Tabs Help

Name: MyStore ☐ Failed ☐ Entrance Locked ☒

Label: ☒ Planned ☐

Attributes Times Failures Controls Exit Statistics Importer Energy C

X-Dimension: 4 ☐

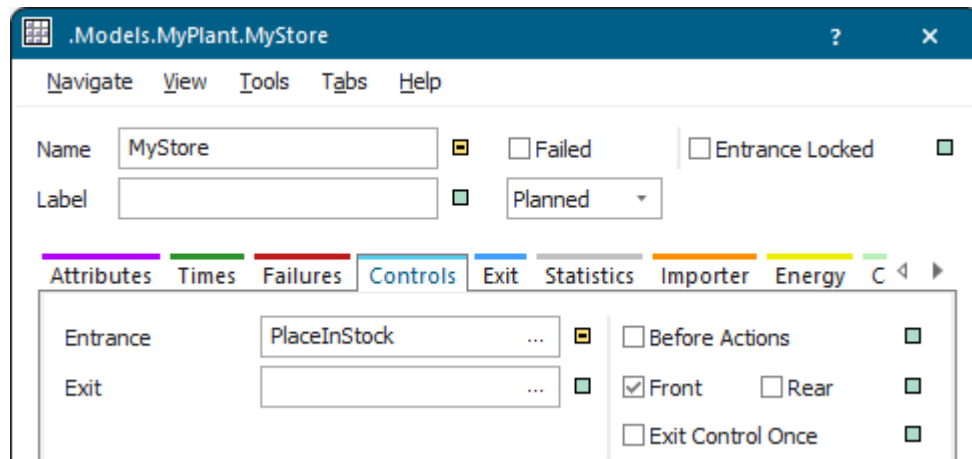
Y-Dimension: 8 ☐

Z-Dimension: 1 ☒

☐ Supermarket ☒

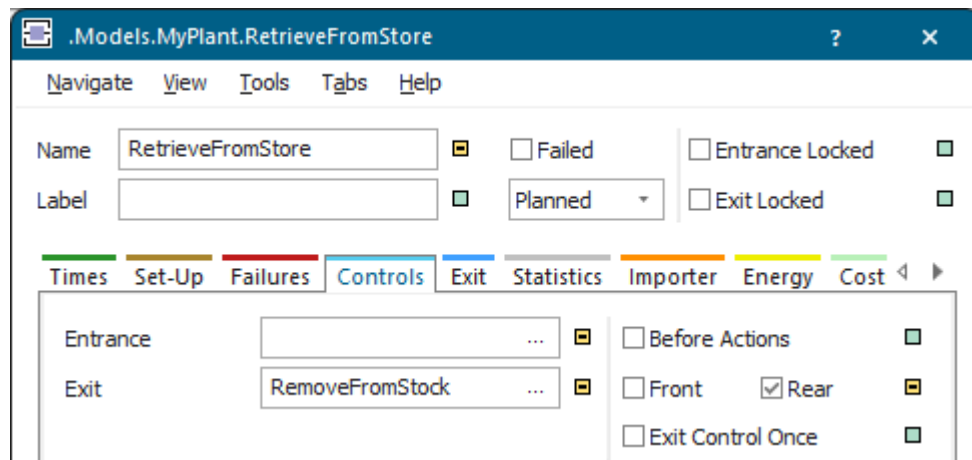
As the *Store* is passive material flow object, you have to program how it handles the parts.

To specify the name of the **Entrance Control**, which registers the parts contained in the *Store* in a data table, click the button and select the name of the method in the dialog that opens. We named our **Entrance Control** *PlaceInStock*.



- To configure the *Station* named *RetrieveFromStore* that receives the parts that are removed from stock, enter and select the settings for the **Exit Control**.

We named our **Exit Control** *RemoveFromStock*. It catches the event after a part has been successfully removed from the *Store*.



Then select the check box **Rear**, so that the control is **Rear**-triggered, i.e., after the part has left the station *RetrieveFromStore*.

Go to [Program the Method that Places Parts into Stock](#)

Back to [Place Parts into Stock and Remove Parts from It](#)

Program the Method that Places Parts into Stock

After configuring the material flow stations in our warehouse model, you have to program the method that places parts into stock of the *Store*.

In our sample model we named this method *placeInStock* and typed in the following source code:

```
// entrance control of the object 'MyStore'
// find next free row in the 'InventoryTable' of 'MyStore'

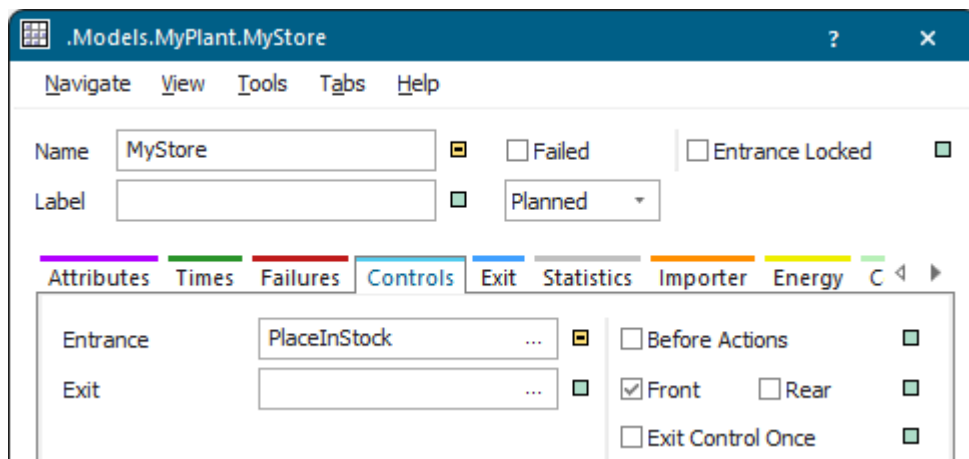
var NextRow : integer
var NextRow := Inventory.ydim + 1

// enter the ID of the new part and the path
// to the part in the 'InventoryTable'
InventoryTable[0,NextRow] := @.id
InventoryTable[1,NextRow] := @

AttemptToRemoveNextPart // name of our Method

// increase the capacity of 'MyStore' if needed
if MyStore.full
    MyStore.xDim := MyStore.xDim + 4
end
```

The object *MyStore* uses this method as the **Entrance Control**.



Back to Place Parts into Stock and Remove Parts from It

Program the Method that Removes Parts from Stock

After configuring the material flow stations in our warehouse model, you have to program the method that removes parts from stock.

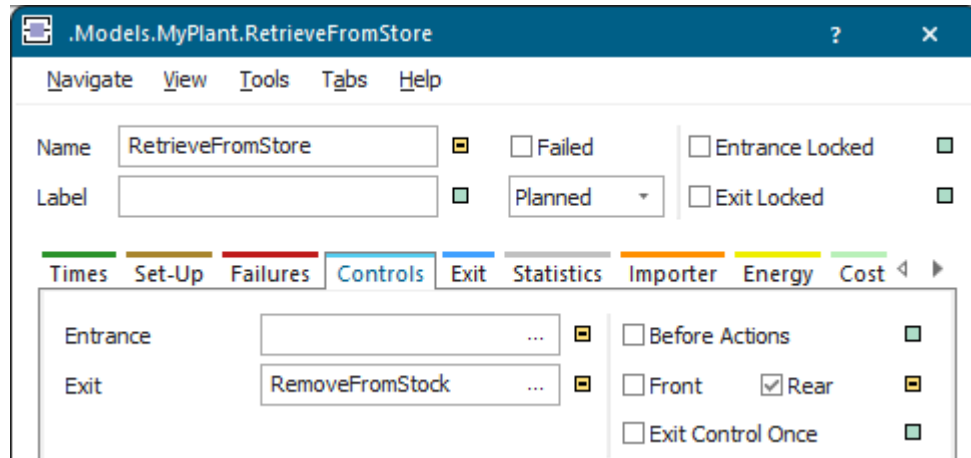
In our sample model we named this method *RemoveFromStock* and entered it as the **Exit Control** of the *Station RetrieveFromStore*. The method *RetrieveFromStore* in turn calls the method *AttemptToRemoveNextPart* which manages the inventory table

We typed in this source code into the **Exit Control** named *RetrieveFromStore*:

```
// rear-triggered exit control of the station 'RetrieveFromStore'
// catches the event after a part has been successfully
// removed from the Store and left the station 'RetrieveFromStore'

AttemptToRemoveNextPart // name of our Method
```

The Station *RetrieveFromStore* uses this method as a **Rear-triggered Exit Control**.

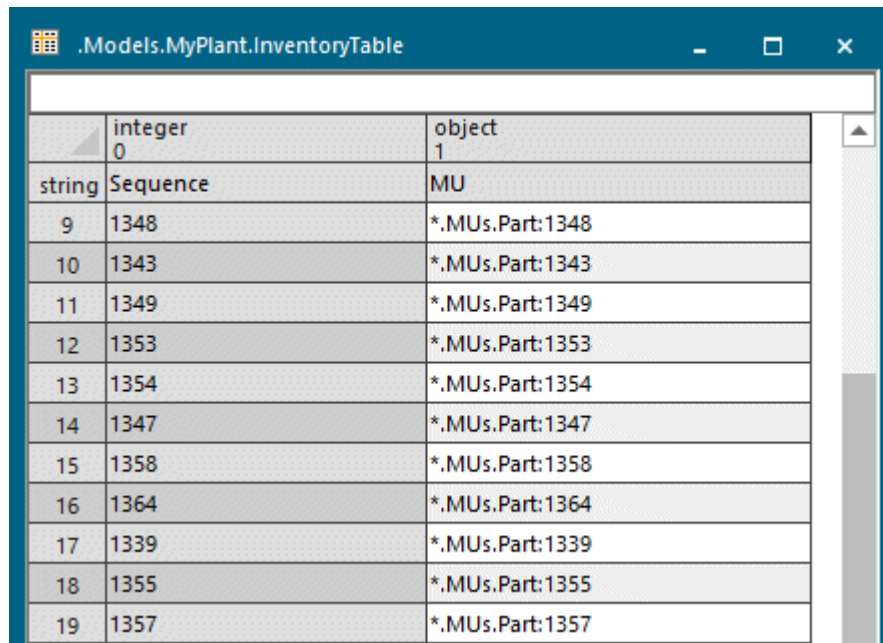


Back to [Place Parts into Stock and Remove Parts from It](#)

Program the Method that Manages the Inventory Table

After configuring the material flow stations in our warehouse model, you have to program the method that manages the inventory table of the *Store MyStore*.

Our inventory table has two columns, whose data types we set in the method below. The column index shows the data types. We also showed the row index and the column index and entered **Sequence** into the first column and **MU** into the second column of the column index.



	integer 0	object 1
string	Sequence	MU
9	1348	*.MUs.Part:1348
10	1343	*.MUs.Part:1343
11	1349	*.MUs.Part:1349
12	1353	*.MUs.Part:1353
13	1354	*.MUs.Part:1354
14	1347	*.MUs.Part:1347
15	1358	*.MUs.Part:1358
16	1364	*.MUs.Part:1364
17	1339	*.MUs.Part:1339
18	1355	*.MUs.Part:1355
19	1357	*.MUs.Part:1357

In our sample model we named the method *attemptToRemoveNextPart* and typed in this source code:

```
// control called by the methods 'placeInStock' and 'removeFromStock'

var sequenceNo : integer
var part       : object

// check if the next part according to the original sequence
// is already located in 'MyStore'
if InventoryTable.getRowNo(#NextNumber)>0
    part := InventoryTable[1,#NextNumber]
    sequenceNo := part.id
end
```

Back to [Place Parts into Stock and Remove Parts from It](#)

Display a Value During the Simulation with a Variable

During the simulation the *Variable*, named *NextNumber* in our sample model, shows the identifier of the last part that exited the *Store* according to the original sequence of parts produced by the *Source* named *Receiving*.

To do so, we extended the source code of the method *attemptToRemoveNextPart*:

```
// control called by the methods 'placeInStock' and 'removeFromStock'

var sequenceNo : integer
var part       : object
```

```
// check if the next part according to the original sequence
// is already located in 'MyStore'
if InventoryTable.getRowNo(#NextNumber)>0
    part := InventoryTable[1,#NextNumber]
    sequenceNo := part.id

// code for displaying the value with the Variable
if sequenceNo = NextNumber AND part.move(AttemptToRemoveNextPart)
    NextNumber := NextNumber + 1
    InventoryTable.cutRow(#part.id)
    print "Retrieved part number: ", part.id
end
end
```

In our *reset Method* we delete the contents of the *inventory table* and the **Variable** again:

```
InventoryTable.delete({0,1}..{*,*})
NextNumber := 1
```

In our model this looks like this:

NextNumber= 1441

InventoryTable

M AttemptToRemoveNextPart

The *Console* shows that the parts really did exit in the restored original sequence.

```
Console
Retrieved part number: 639
Retrieved part number: 640
Retrieved part number: 641
Retrieved part number: 642
Retrieved part number: 643
Retrieved part number: 644
```

Back to [Configure the Stations](#)

Back to [Program the Method that Places Parts into Stock](#)

Back to [Program the Method that Removes Parts from Stock](#)

Back to [Program the Method that Manages the Inventory Table](#)

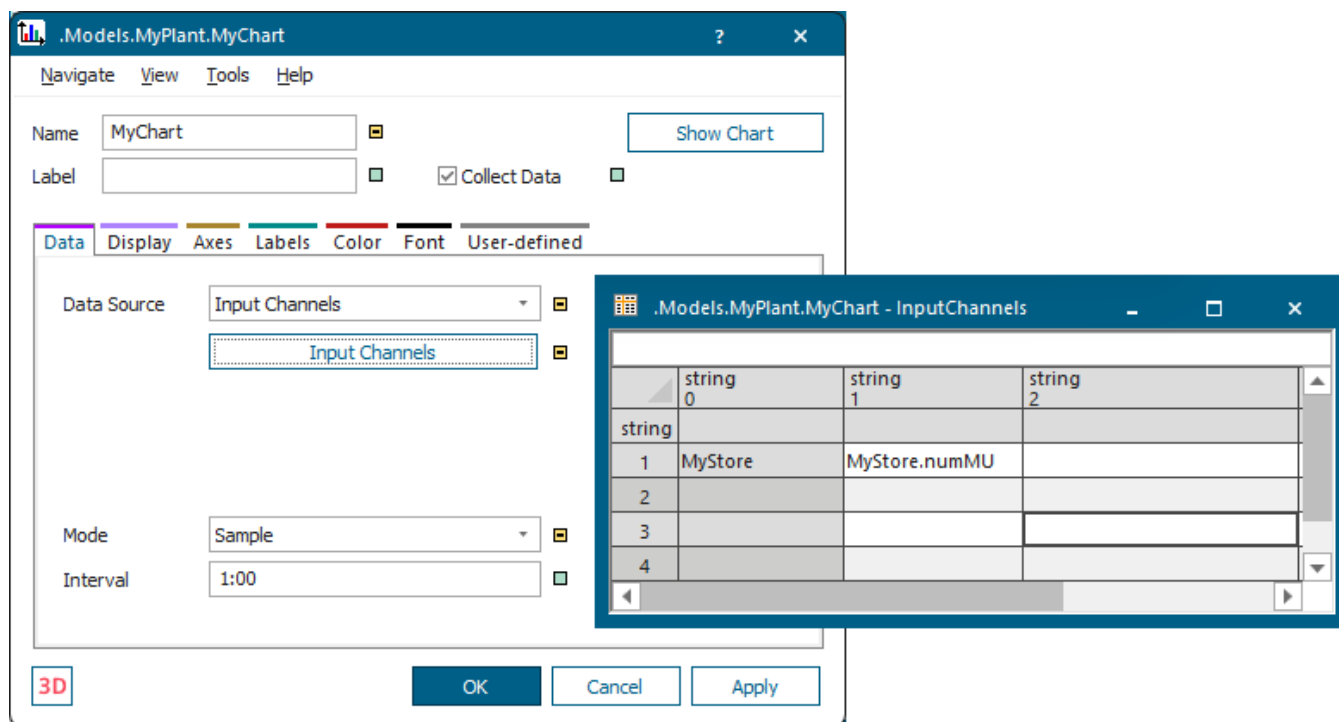
Back to Place Parts into Stock and Remove Parts from It

Visualize the Occupancy of the Store Over Time

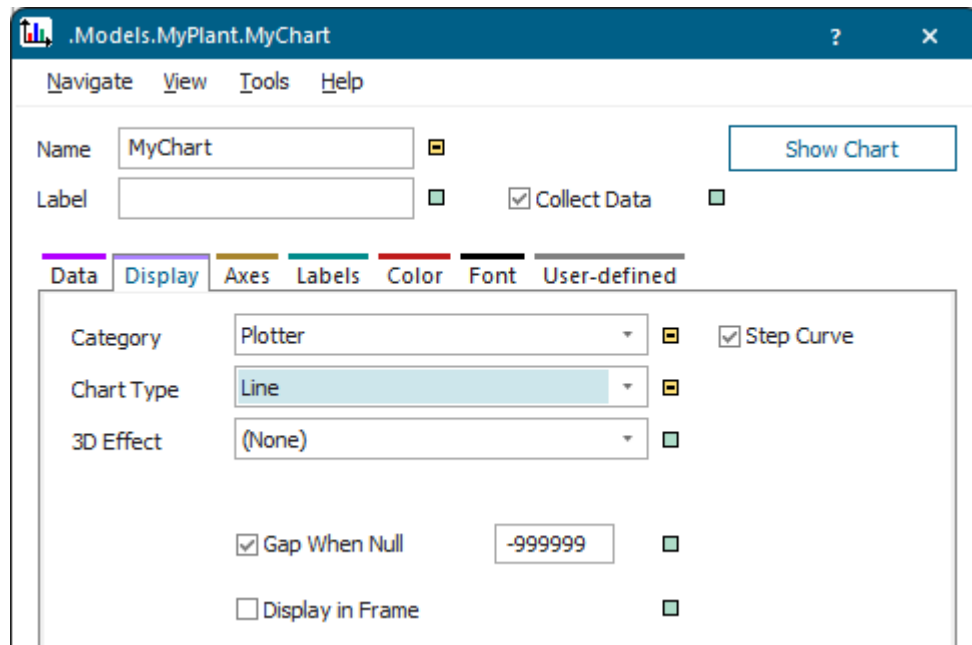
We can use the *Chart* to visualize the occupancy of the *Store* over time.

Insert a **Chart** into the model and select the following settings:

- Select **Input Channels** as the **Data Source**. Then, clear the inheritance check box next to the button **Input Channels** so that it looks like this . Click **Input Channels**. Enter the name of the *Store*, *MyStore*, into row 1. Enter the name of the *Store* and the name of the method whose values you want to display into cell 1 of column 1, *MyStore.numMU* in our case.

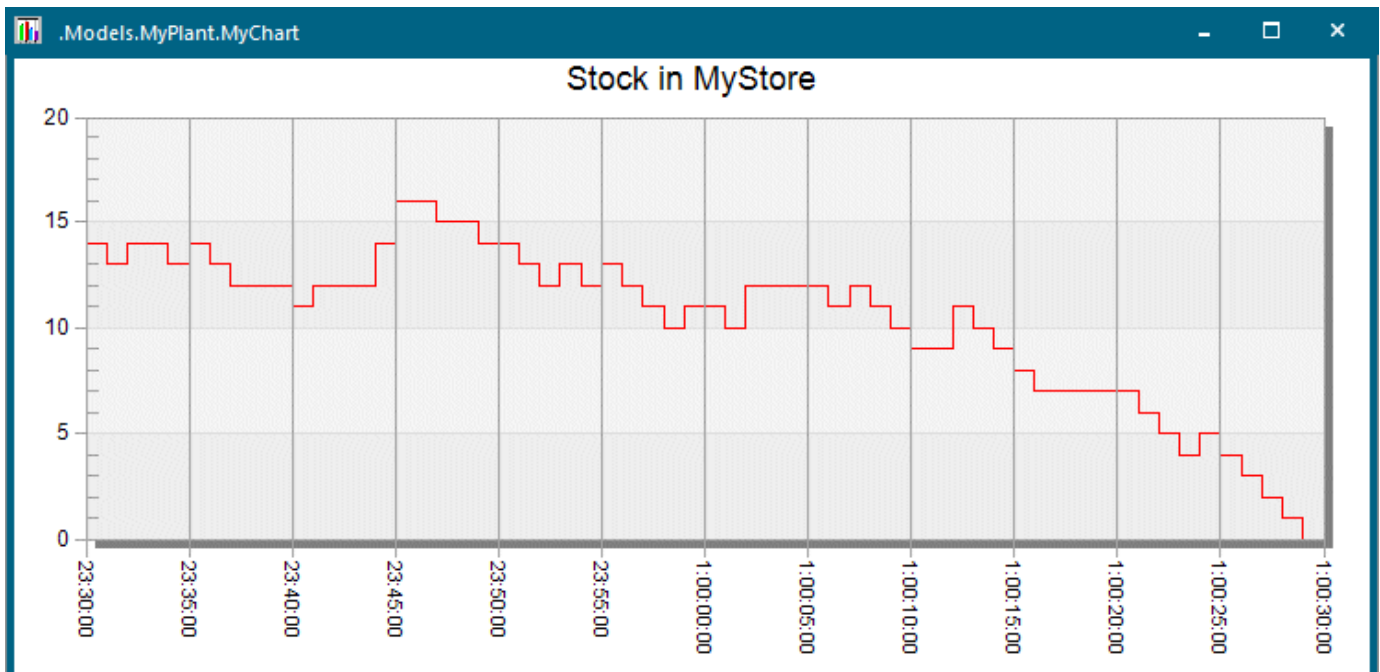


- Click the tab **Display** and select the **Category > Plotter** and the **Chart Type > Line** for displaying the plotted values.



You'll notice that the icon of the *Chart*  changes to the icon of the *Plotter*  in the simulation model when you click **Apply** or **OK**.

- Click the *Plotter* in the model with the right mouse button, select **Show**, start the simulation, and watch how the values develop over time.



See also [Chart](#)